



CICLO FORMATIVO DE GRADO SUPERIOR

DESARROLLO DE APLICACIONES
MULTIPLATAFORMA

Proyecto de Fin de Ciclo

**SIEM Lab MVP: Laboratorio y generación de alertas para
entorno Blue Team**

DAVID RODRIGUEZ IGUAL

Tutor: ARANTXA HERRERA

Curso: CURSO 2025/2026

RESUMEN

El presente proyecto consiste en el desarrollo de **SIEM Lab MVP**, una aplicación web de carácter educativo orientada a simular el flujo básico de un sistema SIEM desde una perspectiva Blue Team. El sistema permite recibir eventos de seguridad simulados, almacenarlos en una base de datos relacional, evaluarlos mediante un motor de reglas propio y generar alertas cuando se cumplen determinadas condiciones de detección.

El proyecto se ha desarrollado como una aplicación propia, evitando basarse en la instalación de plataformas SIEM ya existentes. Para ello se ha implementado un backend con **FastAPI**, una base de datos **PostgreSQL**, modelos gestionados mediante **SQLAlchemy**, migraciones con **Alembic**, un entorno contenerizado con **Docker Compose** y un frontend básico desarrollado con **HTML, CSS y JavaScript**.

La funcionalidad principal se articula en torno al endpoint de ingesta `/ingest`, encargado de recibir eventos, almacenarlos y activar la evaluación mediante reglas. El modelo de datos diferencia entre eventos, reglas y alertas, permitiendo mantener trazabilidad entre el dato recibido, la condición que lo evalúa y la alerta generada. Además, se han incorporado endpoints de consulta enriquecidos para facilitar la visualización de alertas desde la interfaz web.

Durante el desarrollo se han abordado decisiones técnicas relacionadas con el alcance del MVP, la separación de responsabilidades entre componentes, la persistencia de datos, el diseño del motor de reglas y la validación del sistema. El proyecto ha sido probado mediante peticiones manuales a la API, revisión de datos en PostgreSQL, comprobación desde frontend y ejecución de pruebas automatizadas con Pytest.

El resultado final es un laboratorio funcional que representa de forma simplificada el ciclo de ingesta, evaluación, generación y consulta de alertas en un entorno de monitorización defensiva. Aunque no pretende ser una herramienta SIEM de producción, el sistema constituye una base técnica ampliable para futuras mejoras, como la integración con logs reales, autenticación, gestión avanzada de alertas o ampliación del motor de reglas.

ABSTRACT

This project consists of the development of **SIEM Lab MVP**, an educational web application designed to simulate the basic workflow of a SIEM system from a Blue Team perspective. The system receives simulated security events, stores them in a relational database, evaluates them through a custom rule engine and generates alerts when specific detection conditions are met.

The project has been developed as a custom application rather than relying on the installation of existing SIEM platforms. The implementation includes a backend built with **FastAPI**, a **PostgreSQL** database, data models managed with **SQLAlchemy**, database migrations with **Alembic**, a containerized environment using **Docker Compose**, and a basic frontend developed with **HTML, CSS and JavaScript**.

The main functionality is built around the `/ingest` endpoint, which receives events, stores them and triggers rule evaluation. The data model separates events, rules and alerts, making it possible to trace each alert back to the original event and the rule that generated it. Enriched API endpoints were also implemented to support alert visualization from the web interface.

During development, several technical decisions were addressed, including MVP scope definition, component responsibility separation, data persistence, rule engine design and system validation. The project was tested through manual API requests, PostgreSQL data inspection, frontend verification and automated tests using Pytest.

The final result is a functional laboratory that provides a simplified representation of the event ingestion, evaluation, alert generation and alert consultation cycle in a defensive monitoring environment. Although it is not intended to be a production SIEM tool, the system provides an extensible technical foundation for future improvements such as real log integration, authentication, advanced alert management and rule engine expansion.

AGRADECIMIENTOS

Agradezco al tutor del proyecto su orientación y seguimiento durante la elaboración de este trabajo, así como al profesorado del ciclo formativo por los conocimientos transmitidos a lo largo de la formación.

También quiero agradecer al centro educativo los recursos y el acompañamiento proporcionados durante el desarrollo del proyecto.

Finalmente, agradezco a mi familia y personas cercanas su apoyo, comprensión y motivación durante todo el proceso.

Índice

1. Introducción	6
1.1 Objetivos	6
1.2 Contextualización	7
1.3 Justificación	8
2. Gestión del proyecto	9
2.1 Metodología de trabajo	9
2.2 Planificación	10
3. Herramientas y tecnologías utilizadas	11
3.1 Entorno de desarrollo	11
3.2 Backend y API	12
3.3 Base de datos.....	13
3.4 Contenerización.....	13
3.5 Frontend	14
3.6 Validación y documentación	15
4. Desarrollo del proyecto	16
4.1 Arquitectura general del sistema.....	16
4.2 Modelo de datos	18
4.3 API de ingesta	19
4.4 Motor de reglas.....	21
4.5 Gestión de alertas.....	23
4.6 Frontend	24

4.7 Problemas técnicos y decisiones adoptadas.....	25
5. Pruebas y resultado	26
5.1 Pruebas funcionales del entorno y la API	26
Prueba 1. Comprobación de contenedores Docker	27
Prueba 2. Comprobación del endpoint /health.....	28
Prueba 3. Comprobación de Swagger.....	29
Prueba 4. Consulta de métricas	31
Prueba 5. Consulta de reglas	32
5.2 Prueba de ingesta de eventos	34
Prueba 6. Ingesta de evento simulado	34
5.3 Prueba del motor de reglas y generación de alertas	36
Prueba 7. Generación y consulta de alerta	36
5.4 Pruebas de consulta, filtrado y gestión de alertas	39
Prueba 8. Filtros de alertas	39
Prueba 9. Cambio de estado de una alerta	40
5.5 Prueba del frontend	42
Prueba 10. Visualización desde frontend	42
5.6 Pruebas automatizadas	44
Prueba 11. Ejecución de pruebas con Pytest.....	44
5.7 Resultado final.....	46
6. Conclusiones y trabajos futuros	48
6.1 Conclusiones	48
6.2 Limitaciones.....	49
6.3 Líneas futuras.....	50
ANEXOS	53
Anexo I. Diagramas del sistema	53
Figura 1. Diagrama de Gantt de la planificación del proyecto SIEM Lab MVP.....	53
Figura 2. Topología lógica del laboratorio SIEM	53
Figura 3. Diagrama entidad-relación del sistema	53
Figura 4. Flujo de datos del sistema.....	54
Anexo II. Comandos y endpoints principales	54
Comandos principales.....	54
Endpoints principales	54
URLs locales del laboratorio	55

1. Introducción

El presente documento recoge el desarrollo de **SIEM Lab MVP**, una aplicación web educativa orientada a simular el funcionamiento básico de un sistema SIEM desde una perspectiva defensiva o *Blue Team*. El proyecto representa el ciclo principal de tratamiento de eventos de seguridad: recepción de eventos, almacenamiento, evaluación mediante reglas, generación de alertas y consulta de resultados.

Los sistemas SIEM se utilizan en ciberseguridad para centralizar eventos procedentes de distintas fuentes, analizarlos y facilitar la detección de posibles incidentes. En un entorno profesional, estas plataformas pueden incorporar integración con múltiples fuentes de logs, correlación avanzada, usuarios, dashboards, notificaciones e informes. Sin embargo, este proyecto se centra en una versión mínima funcional que permita comprender los fundamentos técnicos de este tipo de sistemas.

La aplicación no pretende sustituir a una herramienta SIEM profesional ni desplegarse en producción. Su objetivo es representar de forma simplificada cómo un evento simulado entra en el sistema, se almacena en una base de datos, se evalúa mediante reglas y, si cumple determinadas condiciones, genera una alerta consultable desde la API o desde una interfaz web básica.

Desde el punto de vista técnico, el proyecto combina backend, base de datos relacional, contenerización, frontend básico y pruebas de validación. A lo largo de la memoria se describen las herramientas utilizadas, la arquitectura del sistema, el modelo de datos, la API de ingesta, el motor de reglas, la gestión de alertas, las pruebas realizadas y las conclusiones obtenidas.

1.1 Objetivos

El objetivo principal del proyecto es desarrollar una aplicación web funcional que simule el flujo básico de un sistema SIEM, permitiendo recibir eventos de seguridad, almacenarlos, evaluarlos mediante reglas y generar alertas consultables desde una API y una interfaz web básica.

Para alcanzar este objetivo, se plantea diseñar una arquitectura de laboratorio separada en backend, base de datos, motor de reglas, herramienta de inspección y frontend. Esta separación permite organizar las responsabilidades de cada componente y facilita la validación del sistema.

También se busca implementar una API REST que actúe como núcleo de la aplicación. Esta API debe permitir recibir eventos mediante un endpoint de ingesta, consultar reglas, generar alertas, filtrar resultados y modificar el estado de las alertas generadas.

Otro objetivo es definir un modelo de datos relacional que diferencie entre eventos, reglas y alertas. Esta estructura permite mantener la trazabilidad entre el evento recibido, la regla activada y la alerta generada.

El proyecto incluye el desarrollo de un motor de reglas propio capaz de evaluar eventos simulados a partir de condiciones como el origen, la severidad, el contenido del mensaje y determinados metadatos. Cuando un evento cumple las condiciones de una regla activa, el sistema debe generar una alerta automáticamente.

Además, se pretende crear una interfaz web básica para visualizar las alertas y contenerizar el entorno mediante Docker Compose, de forma que la API, la base de datos y la herramienta de inspección puedan ejecutarse de manera coordinada y reproducible.

Finalmente, se plantea validar el funcionamiento mediante pruebas manuales, consultas a la API, revisión de datos persistidos y pruebas automatizadas.

1.2 Contextualización

El proyecto se sitúa en el ámbito del desarrollo de aplicaciones aplicado a la ciberseguridad defensiva. La monitorización de eventos y la detección temprana de comportamientos sospechosos son tareas fundamentales para proteger sistemas informáticos. En este contexto, los sistemas SIEM permiten centralizar eventos, analizarlos y generar alertas cuando se detectan condiciones relevantes.

Una plataforma SIEM real puede integrar logs de servidores, aplicaciones, dispositivos de red o sistemas de autenticación, además de incorporar normalización, correlación avanzada, paneles de control, usuarios e informes. Debido a esta complejidad, el proyecto propone una versión reducida que permita comprender su funcionamiento esencial.

SIEM Lab MVP trabaja con eventos simulados, los almacena en una base de datos, los evalúa mediante reglas y genera alertas consultables desde una API y un frontend básico. Desde el punto de vista académico, permite aplicar conocimientos de backend, API REST, base de datos, contenerización, frontend, pruebas y documentación en un contexto relacionado con el ámbito Blue Team.

1.3 Justificación

El desarrollo de **SIEM Lab MVP** se justifica por la necesidad de construir una aplicación que permita comprender de forma práctica cómo funciona el flujo básico de un sistema de monitorización de seguridad. En lugar de instalar una herramienta SIEM ya existente, el proyecto plantea una solución propia, reducida y controlada, que permite aplicar conocimientos de programación, bases de datos, APIs, contenerización y pruebas.

La elección de esta temática responde a la importancia de la detección y gestión de eventos de seguridad. En cualquier sistema informático se generan registros que pueden contener información relevante, como intentos de autenticación, errores, accesos o actividad sospechosa. Reproducir el ciclo de ingesta, evaluación y generación de alertas permite entender mejor uno de los procesos fundamentales del trabajo defensivo.

El proyecto también se justifica desde el punto de vista académico, ya que integra distintas competencias del ciclo de Desarrollo de Aplicaciones Multiplataforma en una única aplicación. El sistema combina backend, base de datos, frontend, Docker, pruebas y documentación, manteniendo un alcance realista.

Por este motivo, se ha optado por construir un MVP centrado en el núcleo funcional: recibir eventos simulados, almacenarlos, evaluarlos mediante reglas y generar alertas consultables. Esta decisión permite obtener un resultado funcional y verificable, evitando una acumulación de funcionalidades incompletas.

2. Gestión del proyecto

La gestión del proyecto se ha organizado de forma incremental, dividiendo el trabajo en fases breves y validables. El objetivo ha sido construir primero el núcleo funcional del sistema y añadir después elementos complementarios como frontend, pruebas, documentación y diagramas.

El proyecto se ha planteado como un **MVP**, por lo que se ha priorizado el flujo esencial: recepción de eventos, almacenamiento, evaluación mediante reglas, generación de alertas y consulta de resultados.

2.1 Metodología de trabajo

La primera fase fue la **investigación y definición del alcance**. Se analizaron las funcionalidades habituales de un SIEM y se seleccionaron únicamente las necesarias para una versión mínima funcional. Se descartaron elementos como autenticación, logs reales, notificaciones o dashboards avanzados para evitar un alcance demasiado amplio.

La segunda fase fue el **diseño técnico**. Se definió la arquitectura general del sistema, el modelo de datos y el flujo principal de funcionamiento. En esta fase se decidió separar la aplicación en backend, base de datos, motor de reglas, frontend y servicios auxiliares.

La tercera fase fue la **preparación del entorno de desarrollo**. Se configuró una máquina virtual Linux llamada siem-lab y se preparó el entorno con Docker Compose para ejecutar la API, PostgreSQL y Adminer.

La cuarta fase fue el **desarrollo del backend**. Se implementó la API REST con FastAPI, incluyendo endpoints de estado, métricas, eventos, reglas, ingesta y alertas.

La quinta fase fue la **implementación de la base de datos y del motor de reglas**. Se creó el modelo relacional con PostgreSQL, SQLAlchemy y Alembic, y se desarrolló la lógica encargada de evaluar eventos y generar alertas.

La sexta fase fue la **gestión de alertas y frontend**. Se añadieron filtros, cambio de estado y endpoints enriquecidos para visualizar alertas desde una interfaz web básica en HTML, CSS y JavaScript.

La última fase fue la **validación y documentación**. Se realizaron pruebas con curl, Swagger, Adminer, frontend y Pytest. También se elaboraron las notas técnicas, los diagramas y la memoria del proyecto.

2.2 Planificación

La planificación se ha distribuido en un periodo aproximado de seis meses. Aunque algunas tareas se han solapado, el desarrollo ha seguido una evolución progresiva.

Durante el primer mes se definieron el alcance, los requisitos y el enfoque MVP. Entre el primer y segundo mes se diseñaron la arquitectura general y el modelo de datos. En el segundo mes se preparó el entorno de desarrollo y se configuraron la máquina virtual, el repositorio y Docker Compose.

Entre el segundo y cuarto mes se desarrollaron el backend, la API REST, la base de datos y el motor de reglas. Entre el cuarto y quinto mes se incorporaron la gestión de alertas, los endpoints enriquecidos, el frontend y la contenerización definitiva del entorno.

Durante el quinto y sexto mes se realizaron las pruebas funcionales y automatizadas, la revisión de resultados, los diagramas y la documentación final.

Figura 1. Diagrama de Gantt de la planificación del proyecto SIEM Lab MVP.

Diagrama de Gantt - Planificación del proyecto SIEM Lab MVP

ACTIVIDAD	RESPONSABLE	TIEMPO (MESES)					
		1	2	3	4	5	6
1. Definición del alcance y requisitos	Alumno						
2. Diseño de arquitectura y modelo de datos	Alumno						
3. Preparación del entorno de desarrollo	Alumno						
4. Desarrollo del backend y API REST	Alumno						
5. Implementación del motor de reglas	Alumno						
6. Gestión de alertas y endpoints UI	Alumno						
7. Desarrollo del frontend	Alumno						
8. Contenerización con Docker Compose	Alumno						
9. Pruebas funcionales y automatizadas	Alumno						
10. Documentación, diagramas y memoria	Alumno						
11. Revisión final y entrega	Alumno / Tutor						

3. Herramientas y tecnologías utilizadas

Para el desarrollo de **SIEM Lab MVP** se han utilizado herramientas orientadas al desarrollo backend, persistencia de datos, contenerización, frontend, validación y documentación. La selección tecnológica se ha realizado buscando tres objetivos principales: construir una aplicación funcional, mantener una arquitectura clara por componentes y facilitar la reproducción del entorno de trabajo.

El proyecto no se basa en la instalación de una plataforma SIEM ya existente, sino en el desarrollo de una aplicación propia. Por este motivo, las herramientas elegidas debían permitir implementar una API REST, almacenar información de forma estructurada, ejecutar servicios de manera aislada, validar el funcionamiento del sistema y documentar el proceso de desarrollo.

3.1 Entorno de desarrollo

El proyecto se ha desarrollado dentro de una máquina virtual Linux llamada siem-lab, ejecutada sobre **VirtualBox**. El uso de una máquina virtual permitió aislar el entorno de desarrollo del sistema anfitrión y trabajar con una configuración controlada. Esta

decisión fue útil para instalar dependencias, levantar contenedores y realizar pruebas sin afectar al equipo principal.

Como editor de código se ha utilizado **Visual Studio Code**, junto con la terminal Linux para ejecutar comandos de Docker, migraciones, pruebas y validaciones manuales. El uso de terminal fue especialmente importante porque gran parte del proyecto se validó mediante comandos curl, ejecución de contenedores, consulta de logs y pruebas automatizadas.

El control de versiones se ha gestionado con **Git**, y el repositorio remoto se ha alojado en **GitHub**. Esto permitió mantener un historial de cambios, revisar el estado del proyecto y disponer de una entrega organizada del código fuente. Git también facilitó controlar los cambios en archivos importantes como el backend, la configuración de Docker, el frontend y la documentación del repositorio.

3.2 Backend y API

El backend se ha desarrollado con **Python 3.12** y **FastAPI**. Python se ha utilizado como lenguaje principal por su claridad y por la disponibilidad de librerías adecuadas para construir APIs, gestionar bases de datos y ejecutar pruebas.

FastAPI se ha utilizado para implementar la API REST del sistema. Esta tecnología permite definir rutas, validar datos de entrada, estructurar respuestas JSON y generar automáticamente documentación interactiva mediante Swagger. Esto resultó especialmente útil durante el desarrollo, ya que permitió probar endpoints como /health, /metrics, /ingest, /rules y /alerts/ui directamente desde el navegador.

La API actúa como núcleo del sistema. A través de ella se reciben eventos, se consultan reglas, se generan alertas, se aplican filtros y se modifican estados. Esta separación permite que el frontend no acceda directamente a la base de datos, sino que consuma información a través de endpoints controlados.

Para ejecutar la aplicación se ha utilizado **Uvicorn**, un servidor ASGI ligero compatible con FastAPI. Uvicorn permite servir la API en el puerto 8000, exponiendo los endpoints necesarios para la ingesta, consulta y validación del sistema.

3.3 Base de datos

La persistencia se ha implementado con **PostgreSQL 16**, una base de datos relacional adecuada para almacenar información estructurada y mantener relaciones entre entidades. En el proyecto se utiliza para almacenar eventos, reglas y alertas.

La elección de PostgreSQL permite representar correctamente el modelo principal del sistema. Un evento representa un dato recibido, una regla define las condiciones de detección y una alerta representa el resultado generado cuando un evento coincide con una regla activa. Esta separación permite mantener trazabilidad entre el evento original, la regla aplicada y la alerta generada.

Para trabajar con la base de datos desde Python se ha utilizado **SQLAlchemy**. Esta herramienta permite definir modelos que representan las tablas del sistema y trabajar con registros mediante código Python. En el proyecto se utilizan principalmente los modelos Event, Rule y Alert.

La gestión de cambios en el esquema de base de datos se ha realizado con **Alembic**. Las migraciones permiten crear o modificar tablas de forma controlada mediante comandos, evitando depender de cambios manuales en la base de datos. Esto facilita reproducir el proyecto en otro entorno y mantener una evolución ordenada del modelo.

3.4 Contenerización

El entorno de ejecución se ha contenerizado mediante **Docker Compose**. Esta herramienta permite definir y levantar varios servicios de forma coordinada.

Los servicios principales del proyecto son:

siem-api → backend FastAPI

siem-db → base de datos PostgreSQL

siem-adminer → herramienta web para inspeccionar PostgreSQL

El uso de Docker Compose aporta varias ventajas. En primer lugar, permite separar los componentes del sistema en servicios independientes. En segundo lugar, facilita levantar el entorno completo con un único comando. En tercer lugar, reduce la dependencia de instalaciones locales, ya que PostgreSQL y Adminer se ejecutan dentro de contenedores.

También se ha trabajado con variables de entorno y archivos `.env`, utilizando `.env.example` como plantilla de configuración. Esta práctica permite documentar las variables necesarias sin exponer credenciales reales en el repositorio.

Durante el desarrollo se comprobó también la importancia de los volúmenes persistentes de Docker. En el caso de PostgreSQL, los datos y credenciales pueden mantenerse aunque el contenedor se reconstruya. Este comportamiento es útil para conservar información, pero también requiere entender que cambiar variables de entorno no siempre modifica una base de datos ya inicializada.

3.5 Frontend

El frontend se ha desarrollado con **HTML, CSS y JavaScript**. Su función es proporcionar una interfaz web básica para visualizar las alertas generadas por el backend.

No se ha utilizado un framework frontend avanzado, ya que el objetivo principal del proyecto no era desarrollar un dashboard complejo, sino demostrar que las alertas generadas podían consultarse visualmente desde navegador. Esta decisión permitió reducir complejidad y mantener el foco en la API, la base de datos y el motor de reglas.

JavaScript se ha utilizado para realizar peticiones HTTP a la API, especialmente al endpoint `/alerts/ui`. Este endpoint devuelve información enriquecida de las alertas,

incluyendo datos del evento asociado y de la regla activada. Gracias a ello, el frontend puede mostrar información útil sin conectarse directamente a PostgreSQL.

El frontend se ha servido localmente mediante el servidor HTTP integrado de Python en el puerto 5173. Esta solución fue suficiente para el alcance del MVP, ya que la interfaz funciona como capa visual de apoyo y no como aplicación independiente desplegada en producción.

3.6 Validación y documentación

Para la validación del sistema se han utilizado varias herramientas. **curl** ha permitido realizar peticiones manuales desde terminal, comprobar respuestas JSON y validar endpoints concretos. Esta herramienta fue especialmente útil para probar `/ingest`, `/metrics`, `/alerts/ui` y el cambio de estado de alertas.

Swagger, generado automáticamente por FastAPI, permitió probar la API desde navegador. Su uso facilitó revisar los endpoints disponibles, comprobar los esquemas de entrada y validar respuestas sin necesidad de escribir siempre comandos manuales.

Adminer se ha utilizado como herramienta auxiliar para inspeccionar PostgreSQL desde navegador. Permitted comprobar visualmente la existencia de tablas y registros, especialmente eventos, reglas y alertas. Esto fue útil para validar que los datos enviados a la API se estaban persistiendo correctamente.

Las pruebas automatizadas se han realizado con **Pytest**. Estas pruebas se ejecutaron dentro del contenedor de la API para asegurar que se utilizaba el entorno correcto del backend. La validación automatizada no cubre todos los casos posibles, pero permite comprobar parte del comportamiento del sistema de forma repetible.

La documentación técnica se ha organizado en **Obsidian**, donde se han preparado notas sobre arquitectura, desarrollo, problemas, pruebas y conclusiones. Los diagramas se han elaborado con **Excalidraw**, representando el modelo entidad-relación, el flujo de datos y la topología lógica del laboratorio.

En conjunto, estas herramientas han permitido construir, ejecutar, validar y documentar una aplicación funcional, manteniendo una separación clara entre desarrollo, persistencia, ejecución, visualización y pruebas.

4. Desarrollo del proyecto

El desarrollo del proyecto se ha centrado en construir una aplicación web funcional que represente el flujo básico de un sistema SIEM. Para ello se ha implementado una arquitectura dividida en componentes, donde cada parte cumple una responsabilidad concreta: recepción de eventos, persistencia, evaluación mediante reglas, generación de alertas, consulta de resultados y visualización desde frontend.

El sistema no se ha planteado como una instalación de herramientas externas, sino como una aplicación propia. Esta decisión ha condicionado el desarrollo, ya que ha sido necesario diseñar el modelo de datos, implementar la API, construir la lógica de detección, definir los endpoints de consulta y preparar un entorno reproducible mediante Docker Compose.

El flujo principal implementado es el siguiente:

evento simulado → API de ingesta → PostgreSQL → motor de reglas → alerta → consulta API/frontend

4.1 Arquitectura general del sistema

La arquitectura del sistema se ha diseñado siguiendo una separación clara de responsabilidades. El backend actúa como núcleo lógico de la aplicación, la base de datos almacena la información persistente, el motor de reglas evalúa eventos y el frontend permite consultar visualmente las alertas generadas.

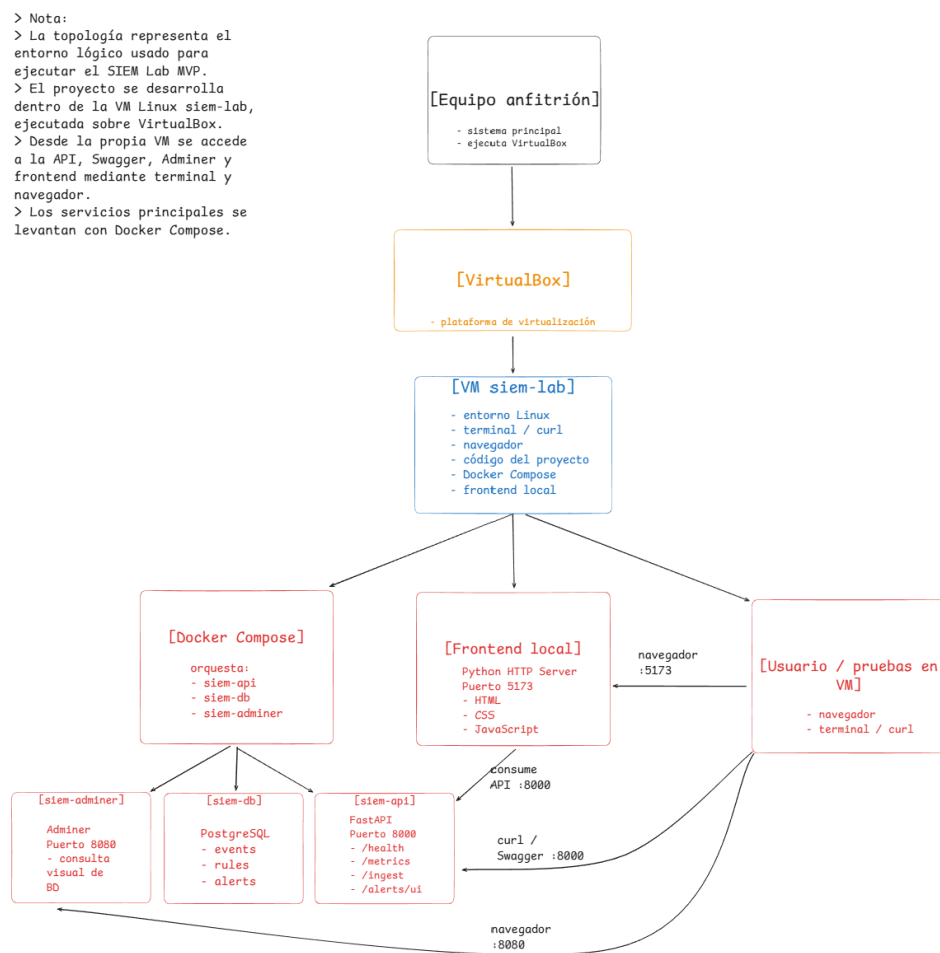
La aplicación se ejecuta dentro de una máquina virtual Linux llamada siem-lab. En esta máquina se levanta el entorno principal mediante Docker Compose, que contiene los servicios siem-api, siem-db y siem-adminer. El frontend se sirve de forma local

mediante el servidor HTTP de Python, fuera de Docker Compose, ya que su función es únicamente mostrar una interfaz básica de consulta.

La API FastAPI expone los endpoints principales del sistema en el puerto 8000. PostgreSQL se utiliza como base de datos relacional para almacenar eventos, reglas y alertas. Adminer se expone en el puerto 8080 para revisar visualmente el contenido de la base de datos. El frontend se sirve en el puerto 5173 y consume los endpoints enriquecidos de la API.

Esta arquitectura permite que el frontend no acceda directamente a la base de datos. Todas las operaciones pasan por la API, que valida peticiones, consulta o modifica datos y aplica la lógica del sistema. Esta separación mejora la organización del proyecto y facilita la validación de cada componente.

Figura 2. Topología lógica del laboratorio SIEM.



4.2 Modelo de datos

El modelo de datos se ha diseñado en torno a tres entidades principales: Event, Rule y Alert. Esta separación permite diferenciar entre el dato recibido por el sistema, la condición que se aplica para evaluarlo y el resultado generado.

Un **evento** representa un registro de seguridad simulado que entra en el sistema. Contiene información como el origen, la severidad, el mensaje y metadatos adicionales. Una **regla** define las condiciones que debe cumplir un evento para generar una alerta. Una **alerta** representa el resultado de aplicar una regla sobre un evento concreto.

La relación principal del modelo es:

Event 1 — 0..N Alert

Rule 1 — 0..N Alert

Esto significa que un evento puede generar una o varias alertas y que una regla puede generar múltiples alertas a lo largo del tiempo. Cada alerta queda asociada tanto al evento que la originó como a la regla que se activó.

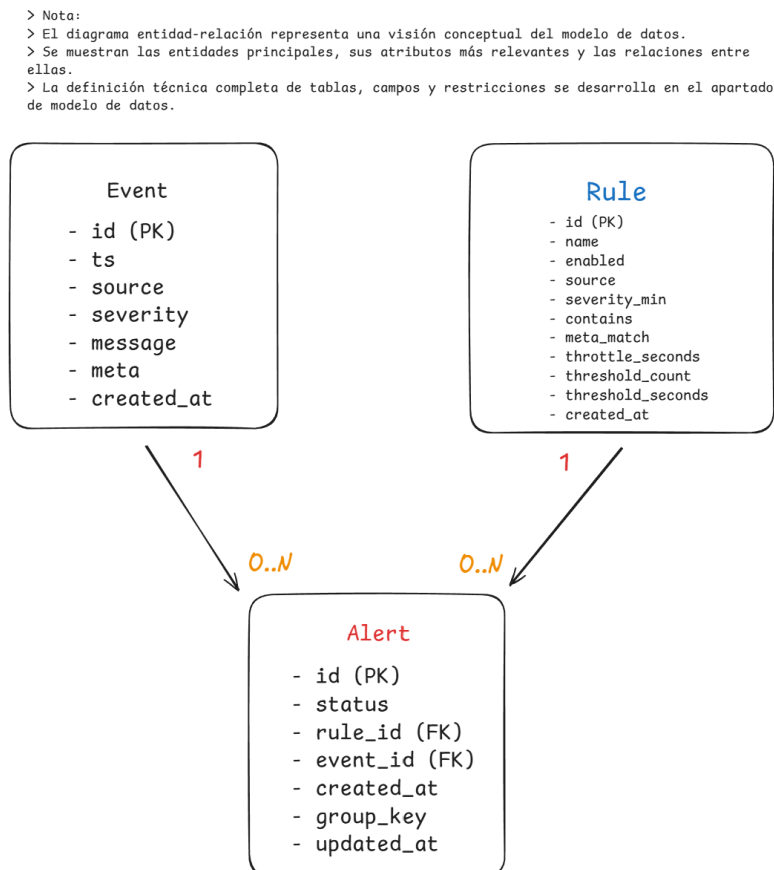
La tabla events almacena los eventos recibidos. Sus campos principales son id, source, severity, message, meta, ts y created_at. El campo meta permite almacenar información adicional en formato flexible, como el host asociado al evento.

La tabla rules almacena las reglas de detección. Incluye campos como name, enabled, source, severity_min, contains, meta_match, threshold_count, threshold_seconds y throttle_seconds. Estos campos permiten definir condiciones de detección simples pero suficientes para el alcance del MVP.

La tabla alerts almacena las alertas generadas. Cada alerta contiene referencias a rule_id y event_id, además de campos como title, status, group_key, created_at y updated_at. El campo status permite representar el estado de revisión de la alerta, con valores como open, ack o closed.

Esta estructura evita duplicar todos los datos del evento dentro de la alerta. En su lugar, la alerta mantiene la referencia al evento original. Cuando el frontend necesita información combinada, la API genera respuestas enriquecidas a través de endpoints específicos como `/alerts/ui`.

Figura 3. Diagrama entidad-relación del sistema.



4.3 API de ingesta

La API de ingesta es el punto de entrada principal del sistema. Se implementa mediante el endpoint:

POST `/ingest`

Este endpoint recibe eventos simulados en formato JSON, valida su estructura, los almacena en PostgreSQL y activa el proceso de evaluación mediante reglas.

Un evento de prueba tiene la siguiente estructura:

```
{  
  "source": "ssh",  
  "severity": 7,  
  "message": "failed password for invalid user demo",  
  "meta": {  
    "host": "demo-host"  
  }  
}
```

El campo source indica el origen del evento. El campo severity representa su gravedad mediante un valor numérico. El campo message contiene el mensaje principal del evento. El campo meta permite incluir información adicional, como el host, usuario o dirección IP.

La lógica del endpoint /ingest sigue este flujo:

1. Recibir petición HTTP.
2. Validar estructura del evento.
3. Crear registro en la tabla events.
4. Obtener reglas activas.
5. Evaluar el evento mediante el motor de reglas.
6. Generar alerta si existe coincidencia.
7. Devolver el evento creado.

Es importante destacar que la respuesta de /ingest devuelve el evento creado, no la alerta generada. La alerta debe comprobarse posteriormente mediante /alerts/ui o mediante las métricas del sistema. Esta separación fue importante durante la validación, ya que evita confundir el identificador del evento con el identificador de la alerta.

El endpoint /ingest se diferencia de /events. Mientras que /events permite trabajar con eventos como registros de base de datos, /ingest representa el flujo funcional del sistema, ya que además de guardar el evento activa el motor de reglas.

4.4 Motor de reglas

El motor de reglas es el componente encargado de evaluar eventos y decidir si deben generar una alerta. Es una de las partes principales del proyecto, ya que transforma el sistema en algo más que una API de almacenamiento.

Cada regla puede contener condiciones como:

```
source
severity_min
contains
meta_match
threshold_count
threshold_seconds
throttle_seconds
```

En la validación principal se utilizó la regla `test_rule_ssh`, configurada para detectar eventos de tipo SSH con severidad suficiente y mensaje relacionado con un fallo de autenticación:

```
source = ssh
severity_min = 5
contains = failed
```

Cuando se envía un evento como:

```
source = ssh
severity = 7
message = failed password for invalid user demo
```

el motor compara cada condición de la regla con el evento recibido. En este caso, el origen coincide, la severidad es superior al mínimo requerido y el mensaje contiene la cadena `failed`. Por tanto, el sistema genera una alerta.

El motor también utiliza el concepto de `group_key`, obtenido a partir de `meta.host`. Este valor permite agrupar eventos relacionados con un mismo host y sirve como base para lógicas como control de duplicados, `threshold` o `throttle`.

En esta versión del proyecto se definió el siguiente comportamiento:

- Las alertas simples pueden generarse aunque no exista `group_key`.
- El `group_key` se obtiene desde `meta.host` cuando está disponible.
- El `throttle` y el control de duplicados dependen del `group_key`.
- Los `thresholds` requieren `group_key` para agrupar eventos correctamente.

Esta decisión permitió mantener una lógica sencilla, pero suficientemente clara para validar el comportamiento del motor. Se priorizó que el sistema fuera comprensible y verificable antes que implementar un motor de correlación avanzado.

Figura 4. Flujo de datos del sistema.

> Nota:
 > El flujo de datos representa una visión funcional del SIEM Lab MVP.
 > En esta versión, los eventos son simulados y se envían mediante el endpoint `/ingest`.
 > El diagrama muestra cómo el sistema recibe el evento, lo almacena, lo evalúa mediante reglas y genera una alerta consultable desde la API o el frontend.



4.5 Gestión de alertas

Cuando el motor de reglas detecta una coincidencia, el sistema crea una alerta en la tabla alerts. Esta alerta queda asociada al evento recibido y a la regla activada mediante los campos event_id y rule_id.

Una alerta generada incluye información como:

id
rule_id
event_id
title
group_key
status
created_at
updated_at

El estado inicial de una alerta es open. Posteriormente puede modificarse mediante el endpoint:

PATCH /alerts/{alert_id}

Los estados contemplados son:

open
ack
closed

Esta funcionalidad permite representar una gestión básica del ciclo de vida de una alerta. Una alerta recién generada queda abierta, puede marcarse como reconocida mediante ack y finalmente cerrarse mediante closed.

Para consultar alertas se implementaron endpoints básicos y enriquecidos. Los endpoints básicos permiten recuperar registros de alerta, mientras que los endpoints enriquecidos combinan información de la alerta, el evento asociado y la regla activada.

El endpoint más relevante para el frontend es:

GET /alerts/ui

Este endpoint devuelve campos como rule_name, event_source, event_severity y event_message, evitando que el frontend tenga que realizar varias consultas o acceder directamente a la base de datos.

También se implementaron filtros para facilitar la consulta de alertas:

status

severity_min

q

limit

offset

Estos filtros permiten consultar alertas por estado, severidad mínima del evento, texto contenido en el mensaje y paginación básica.

4.6 Frontend

El frontend se ha desarrollado con HTML, CSS y JavaScript. Su función es ofrecer una capa visual sencilla para consultar las alertas generadas por el sistema.

La interfaz no contiene lógica de detección ni accede directamente a PostgreSQL. Toda la información se obtiene mediante peticiones HTTP a la API, especialmente al endpoint /alerts/ui.

El flujo del frontend es:

cargar página → solicitar alertas a la API → recibir JSON → renderizar resultados

La información mostrada incluye datos como el identificador de la alerta, estado, regla activada, severidad, origen del evento, mensaje y fecha. Esto permite comprobar visualmente que las alertas generadas por el backend pueden consultarse desde navegador.

El frontend se sirve localmente mediante:

```
python3 -m http.server 5173 -d frontend
```

y se accede desde:

```
http://127.0.0.1:5173/index.html
```

No se ha implementado actualización en tiempo real mediante WebSockets o Server-Sent Events. En esta versión, el usuario puede actualizar manualmente la vista para consultar las alertas más recientes. Esta decisión reduce la complejidad y es coherente con el alcance del MVP.

4.7 Problemas técnicos y decisiones adoptadas

Durante el desarrollo surgieron varios problemas técnicos y decisiones relevantes que condicionaron el resultado final.

Uno de los primeros problemas fue el alcance del proyecto. Un SIEM real incluye muchas funcionalidades avanzadas, como integración con logs reales, usuarios, normalización, correlación compleja, dashboards o notificaciones. Se decidió limitar el proyecto a un MVP centrado en el flujo esencial, evitando desarrollar funcionalidades incompletas.

También aparecieron problemas de virtualización. Inicialmente se trabajó con VMware, pero se migró a VirtualBox para estabilizar el entorno. Posteriormente se resolvieron incidencias relacionadas con módulos de VirtualBox en Kali Linux y configuración gráfica de la máquina virtual.

Otro problema relevante fue la autenticación entre la API y PostgreSQL. El error password authentication failed for user "siem" se debía a que el volumen persistente de PostgreSQL conservaba credenciales anteriores. La solución fue modificar la contraseña directamente dentro de PostgreSQL y reiniciar la API. Este problema permitió comprender mejor la diferencia entre contenedor, imagen y volumen.

Durante el diseño de la API también fue necesario diferenciar entre `/events` y `/ingest`. El primero trabaja con eventos como datos, mientras que el segundo representa el flujo funcional del sistema, ya que guarda el evento y activa el motor de reglas.

Otra decisión importante fue crear endpoints enriquecidos para el frontend. En lugar de duplicar información del evento dentro de la tabla `alerts`, se mantuvo un modelo relacional limpio y se creó `/alerts/ui` para devolver información combinada.

Finalmente, durante la validación se detectó la importancia de separar `event_id` y `alert_id`. La respuesta de `/ingest` devuelve el identificador del evento, mientras que la alerta generada tiene su propio identificador. Esta distinción fue clave para documentar correctamente las pruebas y los resultados.

5. Pruebas y resultado

La fase de pruebas se ha realizado con el objetivo de validar el funcionamiento del sistema desde diferentes niveles: entorno de ejecución, API, base de datos, motor de reglas, gestión de alertas, frontend y pruebas automatizadas.

La validación no se ha limitado a comprobar que los endpoints respondieran correctamente, sino que se ha orientado a verificar el flujo completo de la aplicación:

evento simulado → `/ingest` → PostgreSQL → motor de reglas → alerta → `/alerts/ui` → frontend

Para ello se han utilizado pruebas manuales mediante terminal y curl, comprobaciones desde Swagger, revisión de datos en Adminer, pruebas desde el frontend y ejecución de tests automatizados con Pytest.

5.1 Pruebas funcionales del entorno y la API

Las pruebas funcionales iniciales se realizaron para comprobar que el entorno base del proyecto estaba correctamente levantado antes de validar la lógica específica de ingesta, reglas y alertas. Esta fase permitió confirmar que los servicios principales

estaban activos, que la API respondía correctamente, que existía conexión con PostgreSQL y que la documentación interactiva de FastAPI estaba disponible.

Prueba 1. Comprobación de contenedores Docker

La primera comprobación consistió en revisar el estado de los contenedores definidos en Docker Compose. Esta prueba era necesaria porque el proyecto depende de varios servicios ejecutándose de forma coordinada: la API, la base de datos y la herramienta de inspección de PostgreSQL.

El comando utilizado fue:

```
cd ~/siem-lab/docker  
docker compose ps
```

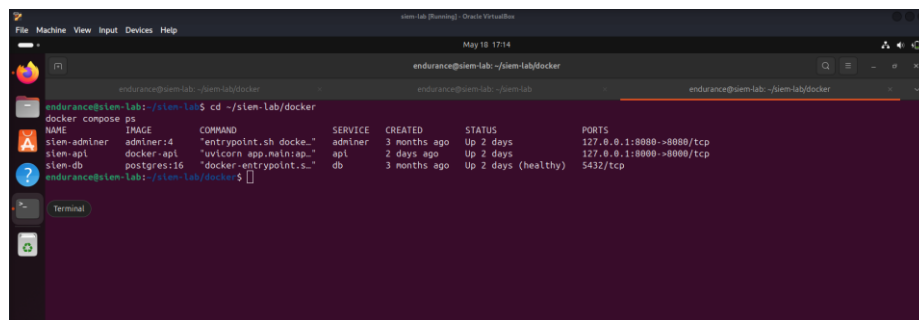
Los servicios esperados eran:

siem-api	→ backend FastAPI
siem-db	→ base de datos PostgreSQL
siem-adminer	→ herramienta web para inspeccionar PostgreSQL

El objetivo de esta prueba era comprobar que los contenedores estuvieran en ejecución y que Docker Compose hubiera levantado correctamente el entorno. Técnicamente, esta comprobación es importante porque el sistema no depende únicamente del backend. Aunque la API esté activa, el flujo completo no funcionaría si PostgreSQL no estuviera disponible o si los servicios no compartieran correctamente la red interna creada por Docker Compose.

La prueba permitió confirmar que el entorno contenerizado estaba operativo y que se podía continuar con las siguientes comprobaciones funcionales.

Figura 5. Comprobación de los contenedores Docker del proyecto.



Tras validar los contenedores, se pudo continuar con la comprobación de la API mediante el endpoint `/health`, ya que el backend y la base de datos se encontraban levantados correctamente.

Prueba 2. Comprobación del endpoint `/health`

La segunda prueba funcional consistió en comprobar el endpoint `/health`, utilizado para verificar el estado general de la API y su conexión con la base de datos.

El comando ejecutado fue:

```
curl http://127.0.0.1:8000/health
```

La respuesta esperada era:

```
{
  "status": "ok",
  "db": "ok"
}
```

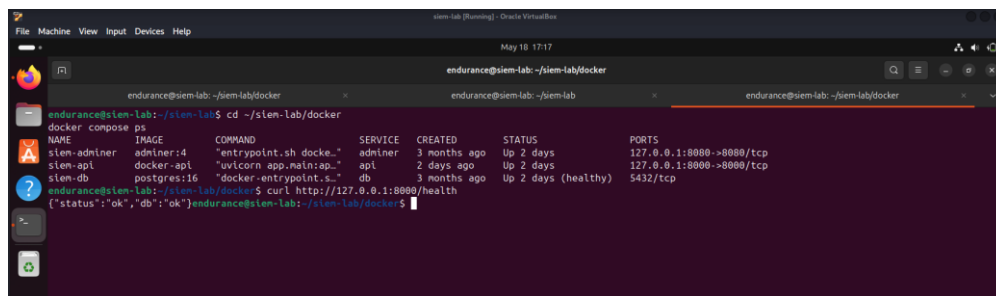
Esta prueba tiene dos niveles de validación. Por un lado, el campo `status` confirma que la aplicación FastAPI está levantada y responde correctamente a las peticiones HTTP. Por otro lado, el campo `db` confirma que la API puede comunicarse con PostgreSQL, lo que permite descartar problemas de conexión entre contenedores, credenciales incorrectas o errores en la configuración de variables de entorno.

Este endpoint resulta especialmente útil como prueba inicial porque permite comprobar rápidamente si el sistema está preparado para ejecutar el resto de validaciones. Si la API respondiera correctamente pero la base de datos fallara, el

flujo de ingesta y generación de alertas no podría funcionar, ya que los eventos, reglas y alertas dependen de la persistencia en PostgreSQL.

La prueba confirmó que tanto la API como la conexión con la base de datos estaban operativas, permitiendo continuar con la revisión de Swagger, métricas y reglas.

Figura 6. Comprobación del endpoint /health de la API.



Prueba 3. Comprobación de Swagger

La tercera prueba consistió en comprobar la documentación interactiva generada automáticamente por FastAPI mediante Swagger. Esta herramienta permite revisar los endpoints disponibles, consultar los esquemas de entrada y salida, y realizar pruebas desde el navegador sin necesidad de escribir manualmente todas las peticiones HTTP.

El acceso se realizó desde la siguiente URL:

<http://127.0.0.1:8000/docs>

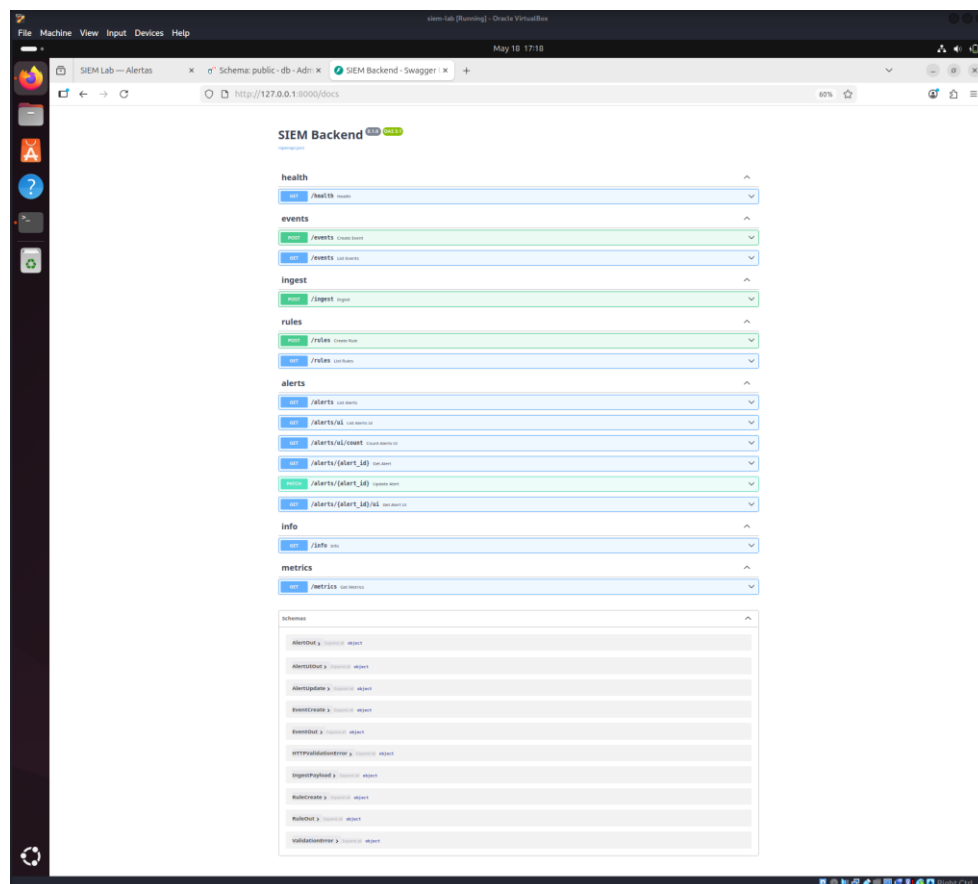
La comprobación de Swagger permitió verificar que la API estaba correctamente expuesta y que FastAPI había generado la documentación OpenAPI del proyecto. En la interfaz se podían consultar los principales grupos de endpoints del sistema, entre ellos los relacionados con estado de la API, métricas, eventos, ingesta, reglas y alertas.

Esta prueba fue útil durante el desarrollo porque permitió validar rápidamente la estructura de la API y comprobar que los endpoints esperados estaban disponibles. También sirvió como apoyo para probar rutas concretas antes de realizar comprobaciones más detalladas desde terminal con curl.

Desde el punto de vista técnico, Swagger no es un componente independiente del sistema, sino una interfaz generada por FastAPI a partir de la definición de la API. Por tanto, su disponibilidad confirma que el backend está correctamente levantado y que las rutas han sido registradas por la aplicación.

La prueba confirmó que la documentación interactiva estaba disponible y que la API podía inspeccionarse desde navegador antes de continuar con las pruebas de métricas, reglas e ingesta.

Figura 7. Documentación interactiva de la API generada con Swagger.



Prueba 4. Consulta de métricas

La cuarta prueba consistió en consultar el endpoint `/metrics`, utilizado para obtener una visión general del estado del sistema mediante contadores agregados.

El comando ejecutado fue:

```
curl -s http://127.0.0.1:8000/metrics | python3 -m json.tool
```

Este endpoint devuelve información resumida sobre los principales elementos almacenados en la base de datos:

```
events_total  
rules_total  
rules_enabled  
alerts_total
```

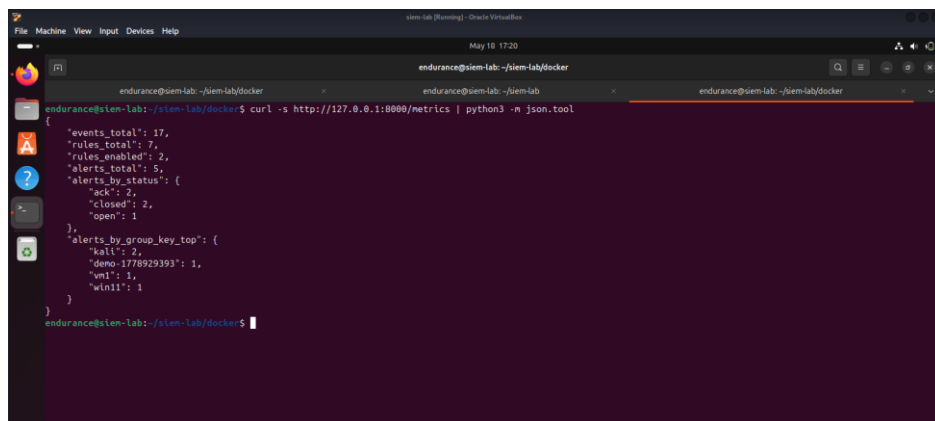
El campo `events_total` indica el número total de eventos registrados en el sistema. El campo `rules_total` muestra el número total de reglas existentes, mientras que `rules_enabled` permite saber cuántas de ellas están activas y disponibles para ser evaluadas por el motor de reglas. Por último, `alerts_total` muestra el número total de alertas generadas.

Esta prueba es relevante porque permite comprobar el estado global del sistema antes y después de ejecutar una ingesta. Por ejemplo, tras enviar un evento válido mediante `/ingest`, el contador de eventos debe aumentar. Si además el evento coincide con una regla activa, también debe aumentar el contador de alertas.

Desde el punto de vista técnico, `/metrics` actúa como una comprobación agregada del funcionamiento del sistema. No sustituye a la revisión detallada de eventos o alertas, pero permite verificar rápidamente si las operaciones realizadas están teniendo efecto sobre la base de datos.

La prueba confirmó que la API podía consultar correctamente los datos persistidos y devolver métricas generales del sistema.

Figura 8. Consulta de métricas generales del sistema mediante `/metrics`.



```

endurance@siem-lab:~/siem-lab/docker$ curl -s http://127.0.0.1:8080/metrics | python3 -m json.tool
{
  "events_total": 17,
  "rules_total": 7,
  "rules_enabled": 2,
  "alerts_total": 5,
  "alerts_by_status": {
    "ack": 2,
    "closed": 2,
    "open": 1
  },
  "alerts_by_group_key_top": {
    "kall": 2,
    "demo-1778929393": 1,
    "win1": 1,
    "win11": 1
  }
}
endurance@siem-lab:~/siem-lab/docker$

```

Prueba 5. Consulta de reglas

La quinta prueba consistió en consultar las reglas configuradas en el sistema mediante el endpoint /rules. Esta comprobación era necesaria antes de validar la ingesta y el motor de reglas, ya que la generación de alertas depende de que exista al menos una regla activa compatible con el evento recibido.

El comando ejecutado fue:

```
curl -s http://127.0.0.1:8000/rules | python3 -m json.tool
```

El objetivo de esta prueba era comprobar que el sistema disponía de reglas almacenadas en PostgreSQL y que la API podía recuperarlas correctamente. Una regla define las condiciones que debe cumplir un evento para generar una alerta. Por tanto, sin reglas activas, el endpoint /ingest podría recibir y guardar eventos, pero no produciría alertas.

En la validación principal se utilizó la regla test_rule_ssh, asociada a la detección de eventos SSH con fallos de autenticación. Las condiciones principales de esta regla eran:

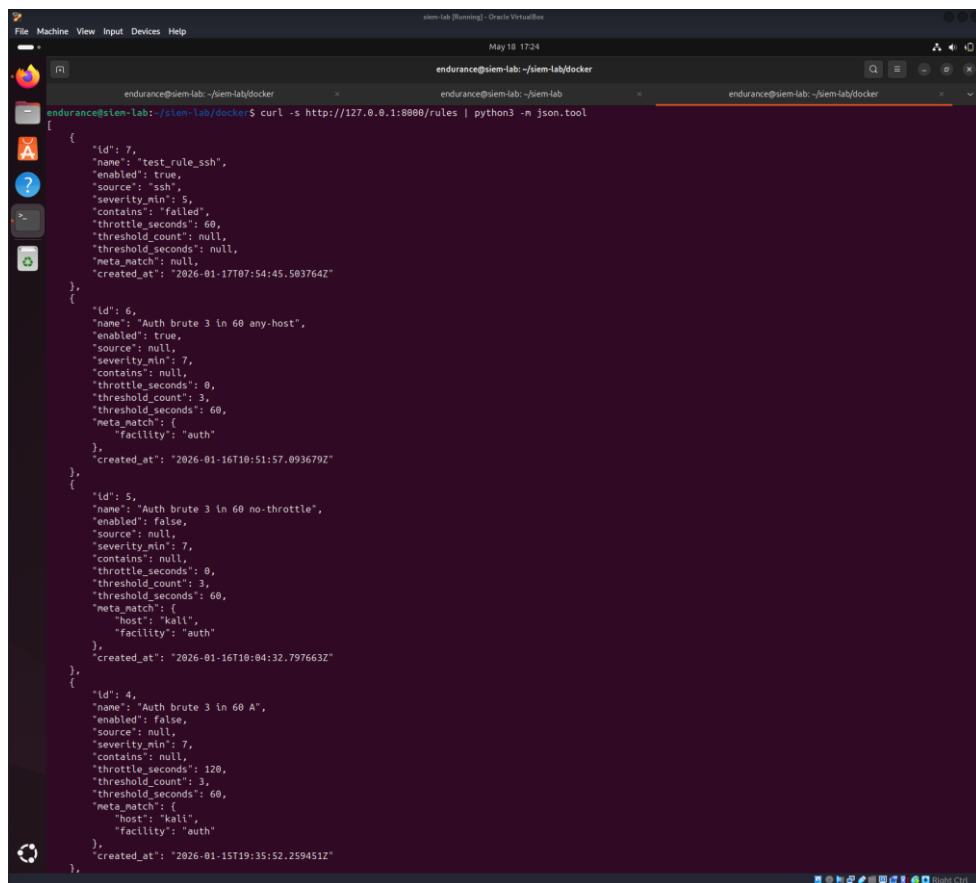
source	=	ssh
severity_min	=	5
contains	=	failed

Desde el punto de vista técnico, esta prueba permite confirmar tres elementos importantes. En primer lugar, que la tabla rules contiene registros. En segundo lugar, que la API puede consultar correctamente esas reglas. En tercer lugar, que existen reglas activas disponibles para el motor de detección.

Esta comprobación también permitió revisar campos como enabled, source, severity_min, contains, threshold_count, threshold_seconds y throttle_seconds. Estos campos definen cómo debe comportarse el motor de reglas ante los eventos recibidos.

La prueba confirmó que las reglas estaban disponibles y que el sistema estaba preparado para evaluar eventos mediante /ingest.

Figura 9. Consulta de reglas configuradas mediante el endpoint /rules.



```
endurance@siem-lab:~/siem-lab/docker$ curl -s http://127.0.0.1:8080/rules | python3 -n json.tool
{
  "id": 7,
  "name": "test_rule_ssh",
  "enabled": true,
  "source": "ssh",
  "severity_min": 5,
  "contains": "failed",
  "throttle_seconds": 60,
  "threshold_count": null,
  "threshold_seconds": null,
  "meta_match": null,
  "created_at": "2026-01-17T07:54:45.503764Z"
},
{
  "id": 6,
  "name": "Auth brute 3 in 60 any-host",
  "enabled": true,
  "source": null,
  "severity_min": 7,
  "contains": null,
  "throttle_seconds": 0,
  "threshold_count": 3,
  "threshold_seconds": 60,
  "meta_match": {
    "facility": "auth"
  },
  "created_at": "2026-01-16T10:51:57.093679Z"
},
{
  "id": 5,
  "name": "Auth brute 3 in 60 no-throttle",
  "enabled": false,
  "source": null,
  "severity_min": 7,
  "contains": null,
  "throttle_seconds": 0,
  "threshold_count": 3,
  "threshold_seconds": 60,
  "meta_match": {
    "host": "kali",
    "facility": "auth"
  },
  "created_at": "2026-01-16T10:04:32.797663Z"
},
{
  "id": 4,
  "name": "Auth brute 3 in 60 A",
  "enabled": false,
  "source": null,
  "severity_min": 7,
  "contains": null,
  "throttle_seconds": 120,
  "threshold_count": 3,
  "threshold_seconds": 60,
  "meta_match": {
    "host": "kali",
    "facility": "auth"
  },
  "created_at": "2026-01-15T19:35:52.259451Z"
}
```

5.2 Prueba de ingesta de eventos

Prueba 6. Ingesta de evento simulado

Una vez comprobado que el entorno, la API, las métricas y las reglas estaban disponibles, se validó el endpoint principal de entrada de eventos:

POST /ingest

Este endpoint representa el inicio del flujo funcional del sistema. A diferencia de una creación simple de registros, /ingest recibe un evento, valida su estructura, lo almacena en PostgreSQL y activa posteriormente la evaluación mediante reglas.

Para la prueba se utilizó un evento simulado relacionado con un fallo de autenticación SSH. El comando ejecutado generó un valor dinámico para el campo host mediante la variable:

```
HOST="demo-$(date +%s)"
```

El objetivo de este valor dinámico era identificar mejor cada prueba y evitar interferencias con eventos anteriores que pudieran compartir el mismo origen lógico.

El evento enviado incluía los campos principales del modelo de ingesta:

```
{  
  "source": "ssh",  
  "severity": 7,  
  "message": "failed password for invalid user demo",  
  "meta": {  
    "host": "demo-1779119427"  
  }  
}
```

El campo source identifica el origen lógico del evento, en este caso ssh. El campo severity indica la gravedad asignada al evento. El campo message contiene el texto que posteriormente puede ser evaluado por el motor de reglas. Por último, el campo

meta permite incluir información adicional en formato flexible; en esta prueba se utilizó para guardar el host asociado al evento.

La respuesta obtenida fue la siguiente:

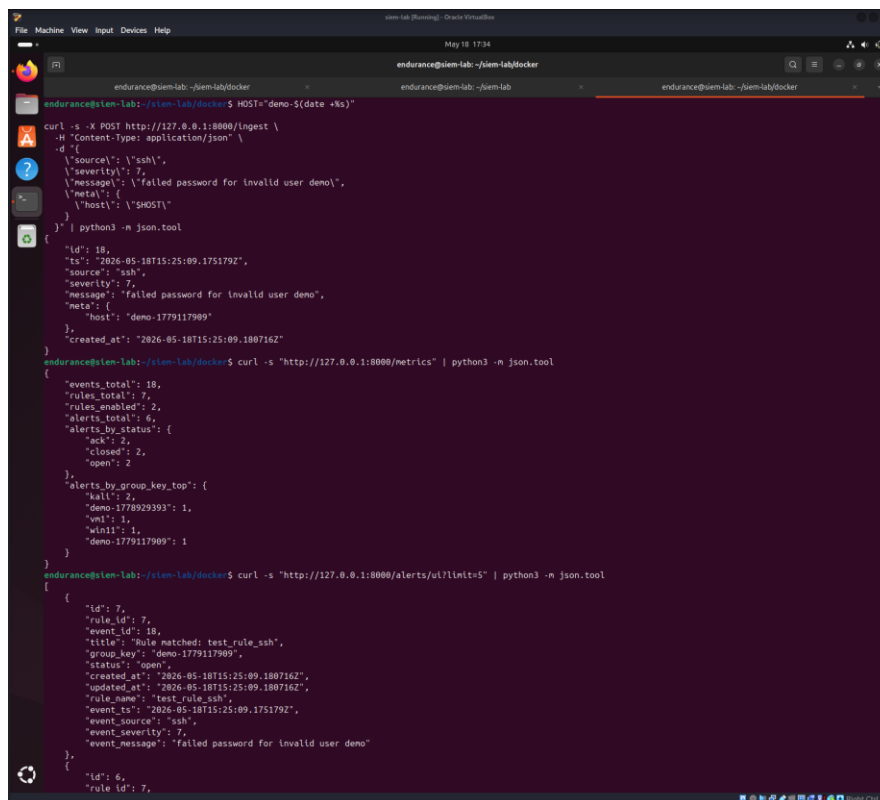
```
{
  "id": 19,
  "ts": "2026-05-18T15:50:27.312118Z",
  "source": "ssh",
  "severity": 7,
  "message": "failed password for invalid user demo",
  "meta": {
    "host": "demo-1779119427"
  },
  "created_at": "2026-05-18T15:50:27.313718Z"
}
```

Esta respuesta confirma que el evento fue recibido correctamente por la API y almacenado en la tabla events. El identificador id: 19 corresponde al evento creado. Los campos ts y created_at permiten registrar el momento del evento y el momento en que el registro fue creado en el sistema.

Desde el punto de vista técnico, esta prueba valida la recepción y persistencia del evento, pero no demuestra por sí sola la generación de una alerta. La respuesta de /ingest devuelve el evento creado, no la alerta resultante. Para comprobar si el evento ha activado una regla y ha generado una alerta, es necesario consultar posteriormente el endpoint /alerts/ui.

Esta distinción es importante porque en el modelo de datos Event y Alert son entidades diferentes. El evento representa el dato recibido por el sistema, mientras que la alerta representa el resultado de aplicar una regla sobre ese evento.

Figura 10. Respuesta del endpoint /ingest tras enviar un evento SSH simulado.



```

endurance@siem-lab:~/siem-lab/docker$ HOST="demo-${date +%s}"
curl -s -X POST http://127.0.0.1:8000/ingest \
-H "Content-Type: application/json" \
-d '{
  "source": "ssh",
  "severity": 7,
  "message": "failed password for invalid user demo",
  "meta": {
    "host": "demo-1779117909"
  }
}' | python3 -m json.tool
{
  "id": 18,
  "ts": "2026-05-18T15:25:09.175179Z",
  "source": "ssh",
  "severity": 7,
  "message": "failed password for invalid user demo",
  "meta": {
    "host": "demo-1779117909"
  },
  "created_at": "2026-05-18T15:25:09.180716Z"
}
endurance@siem-lab:~/siem-lab/docker$ curl -s "http://127.0.0.1:8000/metrics" | python3 -m json.tool
{
  "events_total": 18,
  "rules_total": 7,
  "rules_enabled": 2,
  "alerts_total": 6,
  "alerts_by_status": {
    "ack": 2,
    "closed": 2,
    "open": 2
  },
  "alerts_by_group_key_top": {
    "kall": 2,
    "demo-1778929393": 1,
    "vni": 1,
    "wlois": 1,
    "demo-1779117909": 1
  }
}
endurance@siem-lab:~/siem-lab/docker$ curl -s "http://127.0.0.1:8000/alerts/ui?limit=5" | python3 -m json.tool
[
  {
    "id": 7,
    "rule_id": 7,
    "event_id": 18,
    "title": "Rule matched: test_rule_ssh",
    "group_key": "demo-1779117909",
    "status": "open",
    "created_at": "2026-05-18T15:25:09.180716Z",
    "updated_at": "2026-05-18T15:25:09.180716Z",
    "rule_name": "test_rule_ssh",
    "event_ts": "2026-05-18T15:25:09.175179Z",
    "event_source": "ssh",
    "event_severity": 7,
    "event_message": "failed password for invalid user demo"
  },
  {
    "id": 6,
    "rule_id": 7,

```

5.3 Prueba del motor de reglas y generación de alertas

Prueba 7. Generación y consulta de alerta

Una vez validada la ingesta del evento, se comprobó que el motor de reglas evaluara correctamente el registro recibido y generara una alerta cuando se cumplieran las condiciones definidas.

La regla utilizada para esta validación fue test_rule_ssh, asociada al identificador rule_id: 7. Esta regla estaba configurada para detectar eventos de origen ssh, con una severidad mínima de 5 y cuyo mensaje contuviera la cadena failed.

El evento utilizado en la prueba cumplía estas condiciones, ya que fue registrado con source: ssh, severity: 7 y el mensaje failed password for invalid user demo. Por tanto, el comportamiento esperado era que el motor de reglas generase una alerta asociada a dicho evento.

La comprobación se realizó consultando el endpoint enriquecido de alertas:

```
curl -s "http://127.0.0.1:8000/alerts/ui?limit=5" | python3 -m json.tool
```

La respuesta mostró como primera entrada la alerta generada por el evento anterior:

```
{
  "id": 8,
  "rule_id": 7,
  "event_id": 19,
  "title": "Rule matched: test_rule_ssh",
  "group_key": "demo-1779119427",
  "status": "open",
  "created_at": "2026-05-18T15:50:27.313718Z",
  "updated_at": "2026-05-18T15:50:27.313718Z",
  "rule_name": "test_rule_ssh",
  "event_ts": "2026-05-18T15:50:27.312118Z",
  "event_source": "ssh",
  "event_severity": 7,
  "event_message": "failed password for invalid user demo"
}
```

La relación validada fue la siguiente:

Evento 19 → Regla test_rule_ssh → Alerta 8

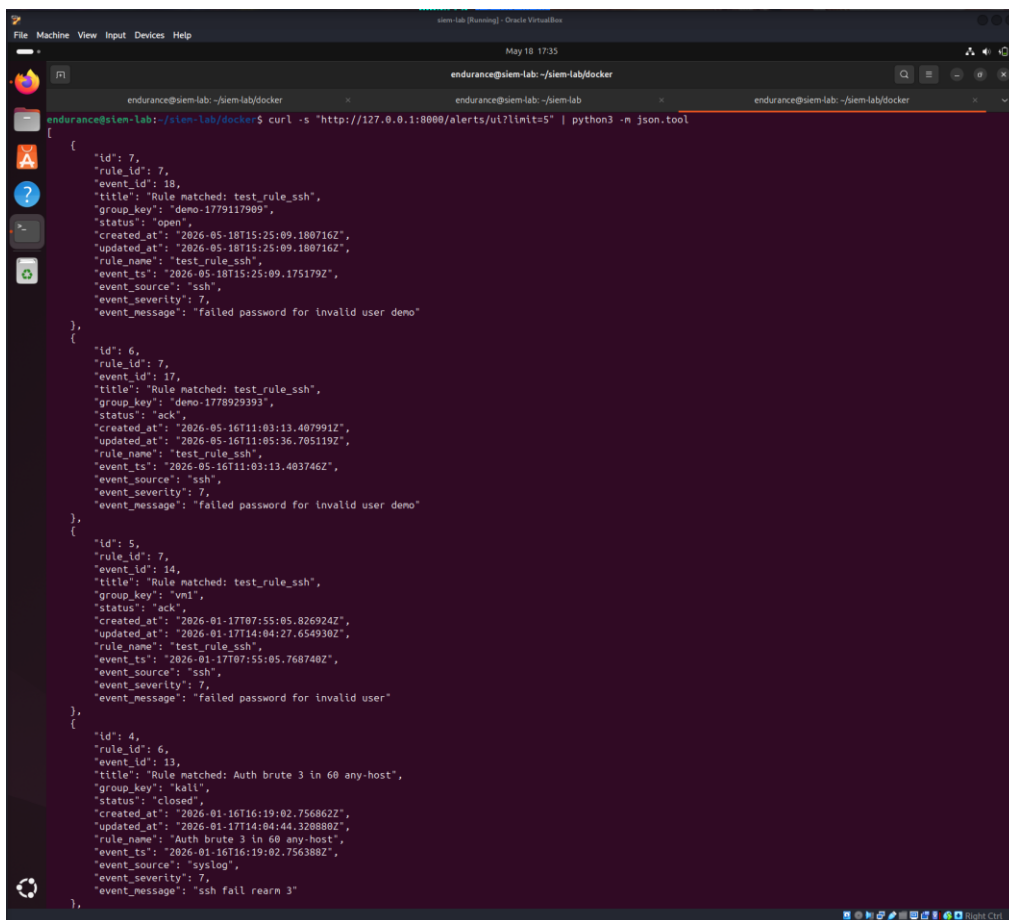
Este resultado confirma que el sistema no se limita a almacenar eventos, sino que aplica una lógica de detección. El motor de reglas consultó las reglas activas, comparó sus condiciones con los campos del evento y creó una alerta cuando encontró una coincidencia.

El campo event_id permite identificar el evento que originó la alerta, mientras que rule_id y rule_name indican qué regla se activó. El campo status aparece inicialmente como open, ya que se trata de una alerta recién generada. Por su parte, group_key toma el valor demo-1779119427, procedente del campo meta.host del evento, y permite agrupar eventos o alertas relacionados con un mismo origen lógico.

Esta prueba también permitió comprobar la utilidad del endpoint `/alerts/ui`. A diferencia de una consulta básica de alertas, este endpoint devuelve información enriquecida combinando datos de la alerta, del evento asociado y de la regla activada. Esto facilita su uso desde el frontend y evita que la interfaz tenga que realizar varias peticiones independientes para reconstruir la información.

En la misma respuesta aparecen alertas anteriores, como las alertas 7, 6 y 5. Esto demuestra que el sistema conserva el histórico de alertas en PostgreSQL y devuelve los resultados ordenados para su consulta. La alerta relevante para esta prueba es la primera, ya que corresponde al evento 19 generado en la prueba de ingesta anterior.

Figura 11. Consulta de `/alerts/ui` mostrando la alerta generada por el motor de reglas.



```
endurance@siem-lab: ~/siem-lab/docker$ curl -s "http://127.0.0.1:8080/alerts/ui?limit=5" | python3 -m json.tool
{
  "id": 7,
  "rule_id": 7,
  "event_id": 18,
  "title": "Rule matched: test_rule_ssh",
  "group_key": "demo-1779117989",
  "status": "open",
  "created_at": "2026-05-18T15:25:09.180716Z",
  "updated_at": "2026-05-18T15:25:09.180716Z",
  "rule_name": "test_rule_ssh",
  "event_ts": "2026-05-18T15:25:09.175179Z",
  "event_source": "ssh",
  "event_severity": 7,
  "event_message": "failed password for invalid user demo"
},
{
  "id": 6,
  "rule_id": 7,
  "event_id": 17,
  "title": "Rule matched: test_rule_ssh",
  "group_key": "demo-1778929393",
  "status": "ack",
  "created_at": "2026-05-16T11:03:13.407991Z",
  "updated_at": "2026-05-16T11:05:36.705119Z",
  "rule_name": "test_rule_ssh",
  "event_ts": "2026-05-16T11:03:13.483746Z",
  "event_source": "ssh",
  "event_severity": 7,
  "event_message": "failed password for invalid user demo"
},
{
  "id": 5,
  "rule_id": 7,
  "event_id": 14,
  "title": "Rule matched: test_rule_ssh",
  "group_key": "vml",
  "status": "ack",
  "created_at": "2026-01-17T07:55:05.826924Z",
  "updated_at": "2026-01-17T14:04:27.654930Z",
  "rule_name": "test_rule_ssh",
  "event_ts": "2026-01-17T07:55:05.768748Z",
  "event_source": "ssh",
  "event_severity": 7,
  "event_message": "failed password for invalid user"
},
{
  "id": 4,
  "rule_id": 6,
  "event_id": 13,
  "title": "Rule matched: Auth brute 3 in 60 any-host",
  "group_key": "kall",
  "status": "closed",
  "created_at": "2026-01-16T16:19:02.756862Z",
  "updated_at": "2026-01-17T14:04:44.328880Z",
  "rule_name": "Auth brute 3 in 60 any-host",
  "event_ts": "2026-01-16T16:19:02.756388Z",
  "event_source": "syslog",
  "event_severity": 7,
  "event_message": "ssh fail rearm 3"
},
}
```

5.4 Pruebas de consulta, filtrado y gestión de alertas

Prueba 8. Filtros de alertas

Tras comprobar que el motor de reglas generaba alertas correctamente, se validaron los filtros disponibles en el endpoint enriquecido de alertas:

GET /alerts/ui

El objetivo de esta prueba era comprobar que la API permitía consultar alertas de forma selectiva, evitando devolver siempre el listado completo de registros. Esta funcionalidad es importante en cualquier sistema de monitorización, ya que el volumen de alertas puede crecer con el tiempo y es necesario poder localizar información concreta.

Se probaron filtros por estado, severidad mínima y búsqueda textual. El filtro por estado permite recuperar alertas según su situación dentro del ciclo de revisión:

```
curl -s "http://127.0.0.1:8000/alerts/ui?status=ack" | python3 -m json.tool
```

El filtro por severidad mínima permite consultar alertas cuyo evento asociado tenga un nivel de severidad igual o superior al valor indicado:

```
curl -s "http://127.0.0.1:8000/alerts/ui?severity_min=7" | python3 -m json.tool
```

La búsqueda textual permite localizar alertas relacionadas con una cadena concreta, como failed, presente en los eventos simulados de autenticación SSH:

```
curl -s "http://127.0.0.1:8000/alerts/ui?q=failed" | python3 -m json.tool
```

Desde el punto de vista técnico, estos filtros no actúan únicamente sobre los campos propios de la alerta, sino también sobre información del evento asociado. Por ejemplo, severity_min depende de la severidad almacenada en el evento, no directamente en la alerta. Esto confirma la utilidad de los endpoints enriquecidos, ya que combinan información procedente de varias entidades del modelo.

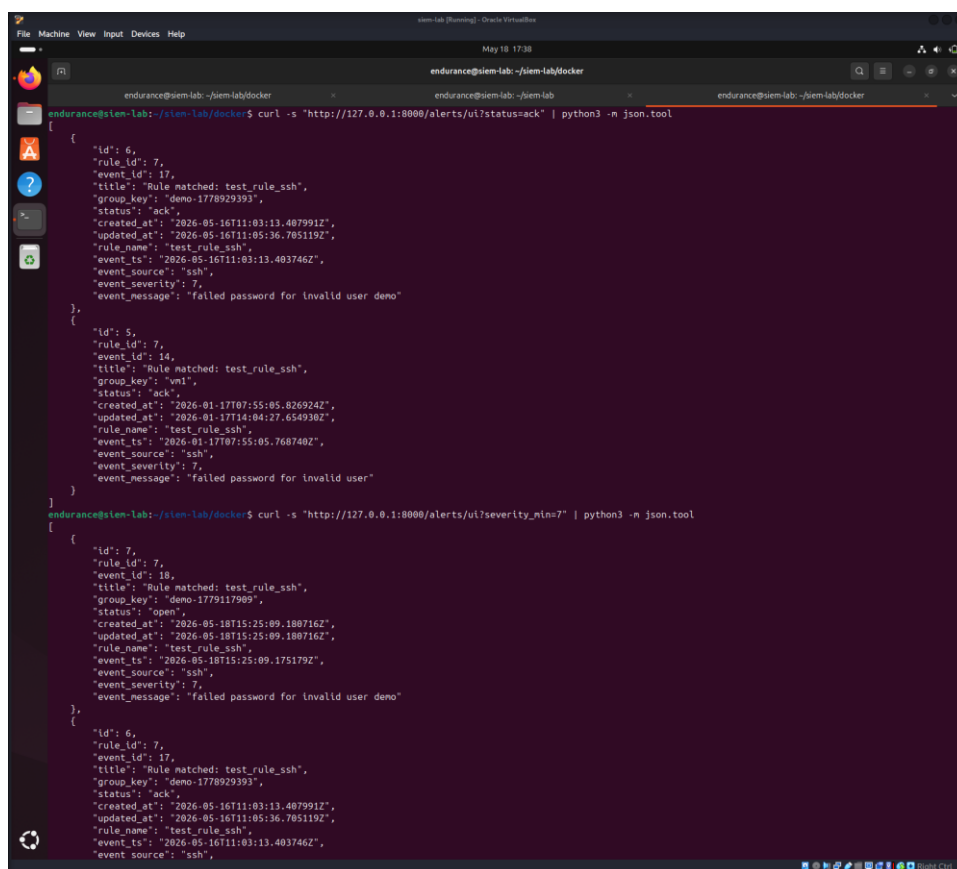
También se validó la limitación de resultados mediante limit, utilizada en consultas como:

```
curl -s "http://127.0.0.1:8000/alerts/ui?limit=5" | python3 -m json.tool
```

Esta opción permite controlar cuántos registros devuelve la API y sirve como base para una posible paginación en versiones futuras.

La prueba confirmó que las alertas podían consultarse aplicando criterios concretos y que la API devolvía resultados coherentes con los filtros utilizados.

Figura 12. Consulta de alertas aplicando filtros mediante /alerts/ui.



```
endurance@siem-lab:~/siem-lab/docker$ curl -s "http://127.0.0.1:8000/alerts/ui?status=ack" | python3 -m json.tool
[
  {
    "id": 6,
    "rule_id": 7,
    "event_id": 17,
    "title": "Rule matched: test_rule_ssh",
    "group_key": "demo-1778929393",
    "status": "ack",
    "created_at": "2026-05-16T11:03:13.407991Z",
    "updated_at": "2026-05-16T11:05:36.705119Z",
    "rule_name": "test_rule_ssh",
    "event_ts": "2026-05-16T11:03:13.403746Z",
    "event_source": "ssh",
    "event_severity": 7,
    "event_message": "failed password for invalid user demo"
  },
  {
    "id": 5,
    "rule_id": 7,
    "event_id": 14,
    "title": "Rule matched: test_rule_ssh",
    "group_key": "vml",
    "status": "ack",
    "created_at": "2026-01-17T07:55:05.826924Z",
    "updated_at": "2026-01-17T14:04:27.654930Z",
    "rule_name": "test_rule_ssh",
    "event_ts": "2026-01-17T07:55:05.768740Z",
    "event_source": "ssh",
    "event_severity": 7,
    "event_message": "failed password for invalid user"
  }
]

endurance@siem-lab:~/siem-lab/docker$ curl -s "http://127.0.0.1:8000/alerts/ui?severity_min=7" | python3 -m json.tool
[
  {
    "id": 7,
    "rule_id": 7,
    "event_id": 18,
    "title": "Rule matched: test_rule_ssh",
    "group_key": "demo-1779117909",
    "status": "open",
    "created_at": "2026-05-18T15:25:09.180716Z",
    "updated_at": "2026-05-18T15:25:09.180716Z",
    "rule_name": "test_rule_ssh",
    "event_ts": "2026-05-18T15:25:09.175179Z",
    "event_source": "ssh",
    "event_severity": 7,
    "event_message": "failed password for invalid user demo"
  },
  {
    "id": 6,
    "rule_id": 7,
    "event_id": 17,
    "title": "Rule matched: test_rule_ssh",
    "group_key": "demo-1778929393",
    "status": "ack",
    "created_at": "2026-05-16T11:03:13.407991Z",
    "updated_at": "2026-05-16T11:05:36.705119Z",
    "rule_name": "test_rule_ssh",
    "event_ts": "2026-05-16T11:03:13.403746Z",
    "event_source": "ssh",
    "event_severity": 7,
    "event_message": "failed password for invalid user demo"
  }
]
```

Prueba 9. Cambio de estado de una alerta

La siguiente prueba consistió en validar la modificación del estado de una alerta. El sistema contempla tres estados principales:

open
ack
closed

Una alerta generada por el motor de reglas aparece inicialmente con estado open. Para simular que la alerta ha sido revisada, se utilizó el endpoint de actualización:

PATCH /alerts/{alert_id}

En la prueba se modificó el estado de una alerta concreta mediante una petición PATCH, enviando en el cuerpo de la petición el nuevo estado:

```
curl -s -X PATCH http://127.0.0.1:8000/alerts/8 \  
-H "Content-Type: application/json" \  
-d '{  
  "status": "ack"  
}' | python3 -m json.tool
```

El objetivo era comprobar que la API podía localizar la alerta por su identificador, modificar el campo status y persistir el cambio en PostgreSQL. La secuencia validada fue:

Alerta 8 → Estado open → Estado ack

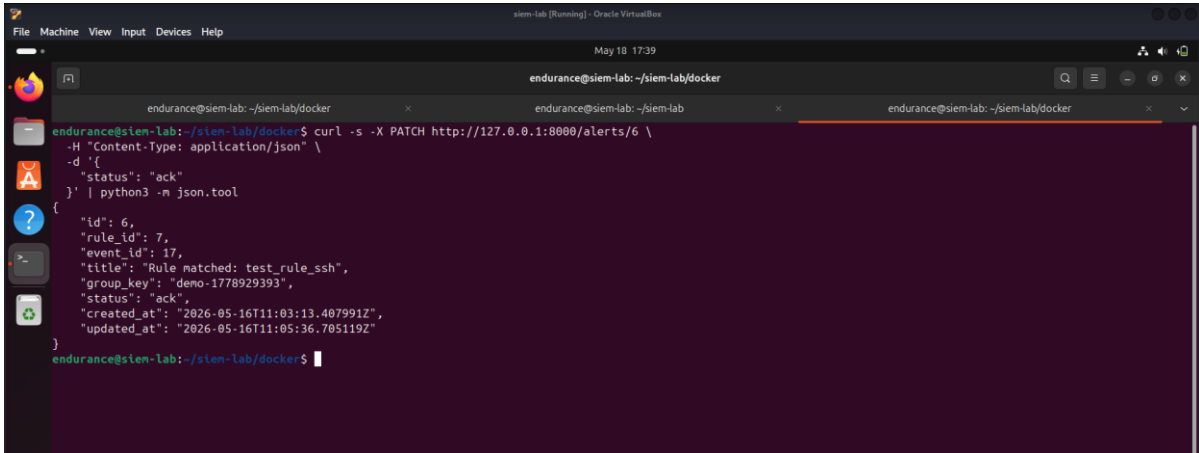
Esta funcionalidad permite representar una gestión básica del ciclo de vida de una alerta. Aunque el sistema no incluye usuarios, asignación de responsables ni comentarios de análisis, el cambio de estado permite simular una revisión inicial propia de una herramienta de monitorización.

Después del cambio, la alerta podía consultarse de nuevo para confirmar que el estado actualizado quedaba registrado. Esta comprobación también podía apoyarse en el filtro status=ack, validando que la alerta modificada aparecía dentro de las alertas reconocidas.

Desde el punto de vista técnico, esta prueba demuestra que la API no solo permite crear y consultar información, sino también actualizar registros existentes de forma

controlada. Además, confirma que el modelo de alertas contempla un estado operativo que puede evolucionar durante el proceso de revisión.

Figura 13. Cambio de estado de una alerta mediante PATCH /alerts/{alert_id}.



```

endurance@siem-lab: ~/siem-lab/docker
endurance@siem-lab:~/siem-lab/docker$ curl -s -X PATCH http://127.0.0.1:8000/alerts/6 \
-d '{
  "status": "ack"
}' | python3 -m json.tool
{
  "id": 6,
  "rule_id": 7,
  "event_id": 17,
  "title": "Rule matched: test_rule_ssh",
  "group_key": "demo-1778929393",
  "status": "ack",
  "created_at": "2026-05-16T11:03:13.407991Z",
  "updated_at": "2026-05-16T11:05:36.705119Z"
}
endurance@siem-lab:~/siem-lab/docker$
  
```

5.5 Prueba del frontend

Prueba 10. Visualización desde frontend

Después de validar la ingesta, la generación de alertas, los filtros y el cambio de estado, se comprobó el funcionamiento del frontend. El objetivo de esta prueba fue verificar que la interfaz web podía consumir la API y mostrar las alertas generadas por el backend.

El frontend se sirvió localmente desde la máquina virtual mediante el servidor HTTP integrado de Python:

```

cd~/siem-lab
python3 -m http.server 5173 -d frontend
  
```

La interfaz se consultó desde el navegador en la siguiente dirección:

<http://127.0.0.1:5173/index.html>

Desde el punto de vista técnico, el frontend no accede directamente a PostgreSQL ni ejecuta lógica de detección. Su función es actuar como consumidor de la API. Para obtener las alertas utiliza principalmente el endpoint enriquecido:

GET /alerts/ui

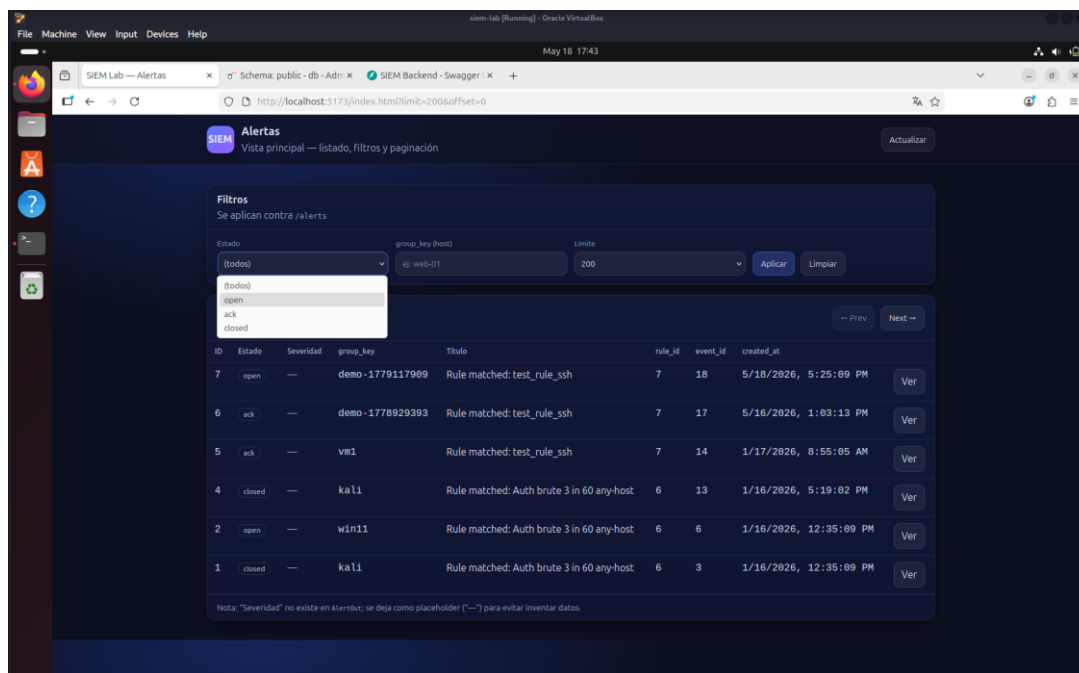
Este endpoint devuelve información combinada de la alerta, del evento asociado y de la regla activada. Gracias a esta respuesta enriquecida, el frontend puede mostrar datos relevantes sin realizar múltiples peticiones independientes ni duplicar información en la base de datos.

La información visualizada incluye campos como el identificador de la alerta, el estado, la regla activada, el origen del evento, la severidad, el mensaje y la fecha de generación. De esta forma, la interfaz permite comprobar visualmente el resultado del flujo completo validado en las pruebas anteriores.

Esta prueba también confirma la separación de responsabilidades entre capas. El backend mantiene la lógica de negocio, el motor de reglas y la comunicación con la base de datos, mientras que el frontend se limita a solicitar datos y renderizarlos en pantalla. Esta separación facilita el mantenimiento del proyecto y evita que la interfaz tenga que conocer directamente la estructura interna de PostgreSQL.

Durante la validación se comprobó que el frontend mostraba correctamente las alertas disponibles en /alerts/ui. También se observó una limitación propia del MVP: la interfaz no incorpora actualización en tiempo real. Por tanto, para ver alertas recién generadas puede ser necesario actualizar manualmente la vista. No se implementaron WebSockets, Server-Sent Events ni polling automático, ya que el objetivo del frontend era ofrecer una visualización básica y no un dashboard avanzado.

Figura 14. Frontend mostrando alertas generadas por el sistema.



5.6 Pruebas automatizadas

Prueba 11. Ejecución de pruebas con Pytest

Además de las pruebas manuales realizadas mediante terminal, Swagger, Adminer y frontend, se ejecutaron pruebas automatizadas con **Pytest**. El objetivo de esta prueba fue comprobar que parte del comportamiento del backend podía validarse de forma repetible, sin depender únicamente de comprobaciones manuales.

Las pruebas se ejecutaron dentro del contenedor de la API, utilizando el siguiente comando:

```
docker compose exec api python -m pytest
```

La decisión de ejecutar Pytest dentro del contenedor siem-api es importante desde el punto de vista técnico. El backend se ejecuta en un entorno containerizado, por lo que las dependencias necesarias para la aplicación están instaladas dentro del contenedor. Ejecutar las pruebas desde el sistema local podría producir errores si el entorno local no contiene las mismas librerías o versiones.

El resultado obtenido fue:

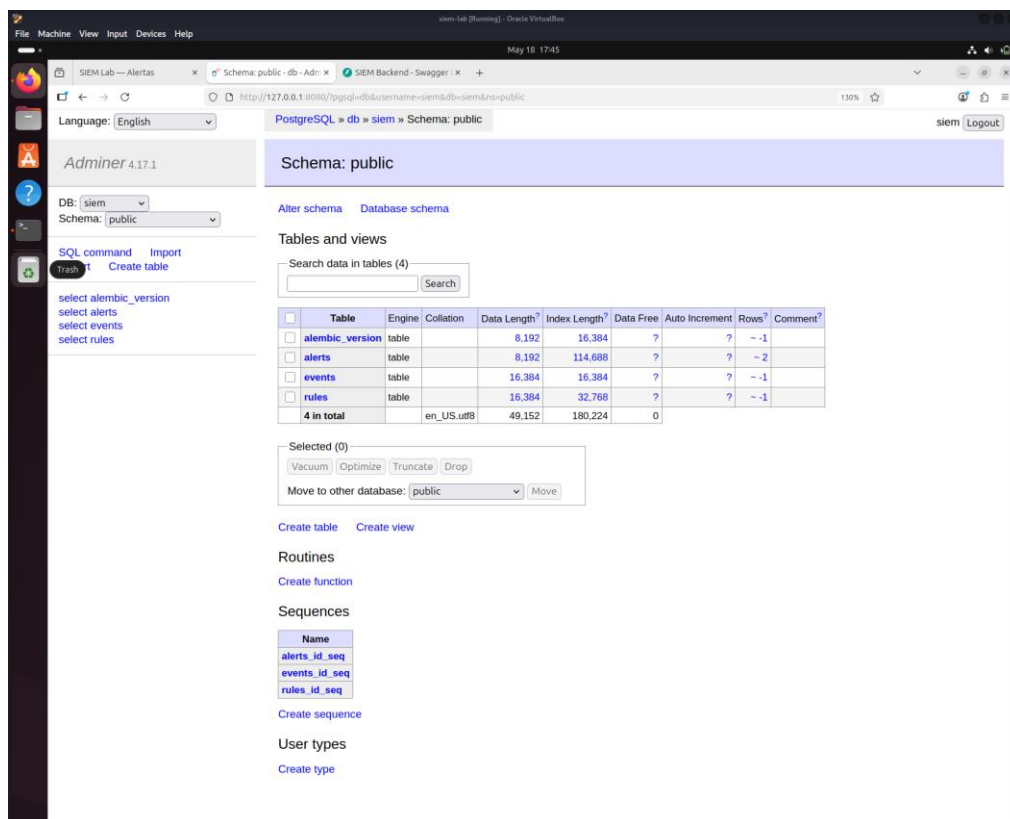
4 passed in 1.00s

Este resultado indica que las cuatro pruebas automatizadas disponibles se ejecutaron correctamente. Aunque el número de pruebas es limitado, su utilidad está en que permiten validar de forma rápida y repetible parte del comportamiento del backend.

Durante el desarrollo se detectó que ejecutar Pytest desde fuera del contenedor podía provocar errores relacionados con dependencias no instaladas, como No module named pytest. Por este motivo, se adoptó como criterio ejecutar las pruebas en el mismo entorno donde se ejecuta la API.

Esta prueba confirma que el proyecto dispone de una base mínima de validación automatizada. No sustituye a las pruebas funcionales manuales, pero las complementa y permite detectar errores básicos después de realizar cambios en el código.

Figura 15. Resultado de las pruebas automatizadas ejecutadas con Pytest.



5.7 Resultado final

El resultado final de las pruebas confirma que el **SIEM Lab MVP** cumple el flujo principal definido para el proyecto. El sistema permite recibir un evento simulado, almacenarlo en PostgreSQL, evaluarlo mediante reglas activas, generar una alerta, consultarla desde la API, filtrarla, modificar su estado y visualizarla desde el frontend.

El flujo completo validado fue:

evento simulado → /ingest → PostgreSQL → motor de reglas → alerta → /alerts/ui → frontend

Las pruebas funcionales iniciales confirmaron que el entorno estaba correctamente levantado mediante Docker Compose. Los servicios siem-api, siem-db y siem-adminer se ejecutaron de forma coordinada, la API respondió correctamente desde /health y la conexión con PostgreSQL quedó validada mediante el valor db: ok.

La prueba de ingesta confirmó que el endpoint /ingest recibía correctamente eventos simulados y los almacenaba en la tabla events. En la validación principal, el evento SSH generado obtuvo el identificador 19, con origen ssh, severidad 7, mensaje de fallo de autenticación y un valor meta.host dinámico.

Posteriormente, la prueba del motor de reglas confirmó que ese evento fue evaluado por la regla test_rule_ssh. Al cumplirse las condiciones de origen, severidad mínima y contenido del mensaje, el sistema generó la alerta 8, asociada al evento 19 y a la regla 7.

La relación principal validada fue:

Evento 19 → Regla 7 test_rule_ssh → Alerta 8

También se comprobó la consulta enriquecida de alertas mediante /alerts/ui. Este endpoint resultó especialmente importante porque devuelve información combinada de la alerta, el evento asociado y la regla activada. Gracias a ello, el frontend puede

mostrar datos útiles sin acceder directamente a PostgreSQL ni realizar varias peticiones independientes.

Las pruebas de consulta, filtrado y gestión confirmaron que las alertas podían recuperarse mediante criterios como estado, severidad mínima y búsqueda textual. Además, se validó el cambio de estado de una alerta mediante PATCH `/alerts/{alert_id}`, representando una gestión básica del ciclo de vida de la alerta.

Desde el frontend se comprobó que las alertas generadas podían visualizarse correctamente en el navegador. La interfaz actuó como consumidor de la API, mostrando la información procesada por el backend. Aunque no se implementó actualización en tiempo real, la visualización manual fue suficiente para validar el objetivo del MVP.

Las pruebas automatizadas con Pytest también se ejecutaron correctamente dentro del contenedor de la API, obteniendo el resultado:

```
4 passed in 1.00s
```

Este resultado no cubre todos los casos posibles del sistema, pero aporta una validación repetible sobre parte del backend y complementa las pruebas manuales realizadas.

Durante la validación se detectaron algunos aspectos importantes. El primero fue la necesidad de diferenciar entre `event_id` y `alert_id`, ya que el evento recibido y la alerta generada son entidades distintas. El segundo fue que una respuesta correcta de `/ingest` valida la creación del evento, pero no demuestra por sí sola que se haya generado una alerta. Para comprobarlo es necesario consultar `/alerts/ui` o revisar las métricas del sistema. El tercero fue que las pruebas automatizadas debían ejecutarse dentro del contenedor de la API para utilizar el entorno correcto de dependencias.

En conjunto, las pruebas realizadas permiten considerar validado el sistema como una versión mínima funcional. El proyecto no reproduce todas las funcionalidades de una plataforma SIEM real, pero sí demuestra de forma práctica el flujo esencial de ingesta, evaluación, generación y consulta de alertas dentro de un entorno controlado.

6. Conclusiones y trabajos futuros

El desarrollo del **SIEM Lab MVP** ha permitido construir una aplicación funcional que representa el flujo básico de un sistema de monitorización defensiva. El sistema recibe eventos simulados, los almacena en PostgreSQL, los evalúa mediante reglas, genera alertas y permite consultarlas desde una API y una interfaz web básica.

El proyecto se ha planteado como un **MVP**, por lo que se ha priorizado un alcance realista frente a una reproducción completa de una plataforma SIEM profesional. Esta decisión ha permitido centrar el trabajo en el flujo principal del sistema y obtener un resultado funcional, validado y documentado.

6.1 Conclusiones

El proyecto ha demostrado que es posible representar de forma simplificada el funcionamiento esencial de un sistema SIEM mediante una aplicación propia. Aunque no pretende ser una herramienta de producción, permite comprender técnicamente cómo un evento puede convertirse en una alerta a partir de reglas de detección.

Desde el punto de vista técnico, se han integrado diferentes áreas del desarrollo de aplicaciones: API REST con FastAPI, base de datos PostgreSQL, modelos con SQLAlchemy, migraciones con Alembic, servicios con Docker Compose y una interfaz web básica con HTML, CSS y JavaScript.

Uno de los elementos clave ha sido el modelo de datos, basado en la separación entre Event, Rule y Alert. Esta estructura permite mantener trazabilidad entre el evento recibido, la regla activada y la alerta generada, evitando duplicar información innecesaria.

El motor de reglas ha aportado la lógica principal del sistema. Gracias a este componente, la aplicación no solo almacena eventos, sino que los evalúa según condiciones como origen, severidad, contenido del mensaje y metadatos. La

validación realizada con la regla `test_rule_ssh` confirmó la generación correcta de alertas a partir de eventos simulados.

Las pruebas realizadas permitieron validar el funcionamiento de extremo a extremo: entorno Docker, conexión entre API y base de datos, ingesta, generación de alertas, filtrado, cambio de estado, frontend y pruebas automatizadas.

En conjunto, el proyecto ha permitido aplicar competencias propias del ciclo de Desarrollo de Aplicaciones Multiplataforma en un contexto relacionado con la ciberseguridad defensiva, obteniendo una aplicación funcional, reproducible y ampliable.

6.2 Limitaciones

El proyecto presenta limitaciones propias de una versión mínima funcional. La principal es el uso de **eventos simulados**, ya que el sistema no recibe logs reales desde servidores, aplicaciones, firewalls o sistemas operativos. Esta decisión permitió controlar mejor las pruebas, pero reduce el realismo del laboratorio.

Otra limitación es la ausencia de **autenticación, roles y permisos**. La API no diferencia perfiles de usuario ni protege endpoints críticos, como la creación de reglas, la ingesta de eventos o el cambio de estado de alertas.

El motor de reglas también tiene un alcance limitado. Permite evaluar condiciones básicas como `source`, `severity_min`, `contains`, `meta_match`, `threshold` o `throttle`, pero no implementa correlación avanzada, reglas compuestas, secuencias temporales complejas ni integración con inteligencia de amenazas.

El frontend cumple una función básica de visualización, pero no incluye gráficos avanzados, gestión de reglas, actualización en tiempo real ni paneles estadísticos. Del mismo modo, la gestión de alertas se limita a estados como `open`, `ack` y `closed`, sin historial de cambios, comentarios, asignación de responsables ni agrupación en incidentes.

El despliegue se ha realizado en un entorno local de laboratorio mediante Docker Compose, por lo que no está preparado para producción. No se han incorporado HTTPS, reverse proxy, gestión avanzada de secretos, backups, alta disponibilidad ni hardening de contenedores.

Finalmente, las pruebas automatizadas tienen un alcance reducido. Aunque Pytest confirma que las pruebas existentes se ejecutan correctamente, no cubren todos los casos posibles del sistema, como pruebas de carga, seguridad, frontend o combinaciones complejas del motor de reglas.

6.3 Líneas futuras

Una primera línea de mejora sería incorporar **fuentes reales de eventos**, como logs de Linux, servicios SSH, servidores web, contenedores Docker o dispositivos de red. Esto permitiría validar el sistema con datos más cercanos a un entorno real.

También podría desarrollarse un **agente básico de envío de eventos**, encargado de leer archivos de log, transformar cada línea relevante en un evento estructurado y enviarlo a la API mediante /ingest.

Otra mejora importante sería añadir una fase de **normalización de eventos**. En la versión actual, todos los eventos tienen una estructura controlada, pero en un entorno real los logs pueden tener formatos muy diferentes. Una capa de normalización permitiría convertirlos a un modelo común antes de aplicar reglas.

También sería recomendable incorporar **autenticación y autorización**, con perfiles diferenciados como administrador, analista o usuario de solo lectura. Esto permitiría proteger endpoints críticos y acercar el proyecto a un entorno más realista.

El motor de reglas podría ampliarse con operadores lógicos más flexibles, reglas compuestas, correlación entre eventos, detección por secuencias temporales y mecanismos para reducir falsos positivos.

La gestión de alertas podría evolucionar hacia un sistema más completo, incorporando historial de cambios, comentarios, asignación de responsables y agrupación de alertas en incidentes.

En el frontend podrían añadirse gráficos, paneles de métricas, filtros avanzados, vista de detalle, gestión de reglas y actualización automática mediante polling, WebSockets o Server-Sent Events.

Por último, sería conveniente reforzar el despliegue y las pruebas automatizadas, incorporando HTTPS, reverse proxy, gestión segura de secretos, backups de PostgreSQL, pruebas de integración, pruebas negativas y validación completa del flujo de ingesta, alerta y consulta.

Estas mejoras permitirían evolucionar el proyecto desde un MVP académico hacia un laboratorio de monitorización defensiva más completo, manteniendo como base la arquitectura ya desarrollada.

BIBLIOGRAFÍA

Alembic. (s. f.). *Alembic documentation*. Recuperado el 21 de mayo de 2026, de <https://alembic.sqlalchemy.org/>

Docker. (s. f.). *Docker documentation*. Recuperado el 21 de mayo de 2026, de <https://docs.docker.com/>

Docker. (s. f.). *Docker Compose overview*. Recuperado el 21 de mayo de 2026, de <https://docs.docker.com/compose/>

Excalidraw. (s. f.). *Excalidraw*. Recuperado el 21 de mayo de 2026, de <https://excalidraw.com/>

FastAPI. (s. f.). *FastAPI documentation*. Recuperado el 21 de mayo de 2026, de <https://fastapi.tiangolo.com/>

Git. (s. f.). *Git documentation*. Recuperado el 21 de mayo de 2026, de <https://git-scm.com/doc>

GitHub. (s. f.). *GitHub Docs*. Recuperado el 21 de mayo de 2026, de <https://docs.github.com/>

MDN Web Docs. (s. f.). *Fetch API*. Recuperado el 21 de mayo de 2026, de https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

MDN Web Docs. (s. f.). *HTML: HyperText Markup Language*. Recuperado el 21 de mayo de 2026, de <https://developer.mozilla.org/en-US/docs/Web/HTML>

MDN Web Docs. (s. f.). *CSS: Cascading Style Sheets*. Recuperado el 21 de mayo de 2026, de <https://developer.mozilla.org/en-US/docs/Web/CSS>

MDN Web Docs. (s. f.). *JavaScript*. Recuperado el 21 de mayo de 2026, de <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

Obsidian. (s. f.). *Obsidian Help*. Recuperado el 21 de mayo de 2026, de <https://help.obsidian.md/>

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. Recuperado el 21 de mayo de 2026, de <https://www.postgresql.org/docs/>

pytest. (s. f.). *pytest documentation*. Recuperado el 21 de mayo de 2026, de <https://docs.pytest.org/>

Python Software Foundation. (s. f.). *Python 3 documentation*. Recuperado el 21 de mayo de 2026, de <https://docs.python.org/3/>

SQLAlchemy. (s. f.). *SQLAlchemy documentation*. Recuperado el 21 de mayo de 2026, de <https://docs.sqlalchemy.org/>

Uvicorn. (s. f.). *Uvicorn documentation*. Recuperado el 21 de mayo de 2026, de <https://www.uvicorn.org/>

ANEXOS

Los anexos recogen información técnica complementaria utilizada durante el desarrollo y validación del proyecto. Se incluyen únicamente los elementos necesarios para apoyar la comprensión del sistema sin repetir el contenido desarrollado en la memoria.

Anexo I. Diagramas del sistema

Figura 1. Diagrama de Gantt de la planificación del proyecto SIEM Lab MVP.

El diagrama muestra una planificación orientativa. Algunas tareas, como pruebas y documentación, se realizaron de forma continua durante el desarrollo, aunque su cierre se concentró en la fase final.

Figura 2. Topología lógica del laboratorio SIEM

La figura muestra la estructura lógica del entorno utilizado para ejecutar el proyecto. El sistema se desarrolla dentro de una máquina virtual Linux llamada siem-lab, ejecutada sobre VirtualBox. Dentro de esta máquina se levantan los servicios principales mediante Docker Compose: API FastAPI, PostgreSQL y Adminer. El frontend se sirve localmente y consume la API desde el puerto 8000.

Figura 3. Diagrama entidad-relación del sistema

La figura representa las entidades principales del modelo de datos: Event, Rule y Alert. Cada alerta queda asociada al evento que la originó y a la regla que se activó, permitiendo mantener trazabilidad entre los datos recibidos y las alertas generadas.

Figura 4. Flujo de datos del sistema

La figura resume el recorrido funcional de la información dentro del sistema: evento simulado, ingesta mediante /ingest, almacenamiento en PostgreSQL, evaluación por el motor de reglas, generación de alerta y consulta desde API o frontend.

Anexo II. Comandos y endpoints principales

Comandos principales

Función	Comando
Levantar servicios	<code>docker compose up -d --build</code>
Ver contenedores	<code>docker compose ps</code>
Aplicar migraciones	<code>docker compose exec api alembic upgrade head</code>
Comprobar API	<code>curl http://127.0.0.1:8000/health</code>
Consultar métricas	<code>curl -s http://127.0.0.1:8000/metrics python3 -m json.tool</code>
Consultar reglas	<code>curl -s http://127.0.0.1:8000/rules python3 -m json.tool</code>
Consultar alertas	<code>curl -s "http://127.0.0.1:8000/alerts/ui?limit=5" python3 -m json.tool</code>
Ejecutar pruebas	<code>docker compose exec api python -m pytest</code>
Servir frontend	<code>python3 -m http.server 5173 -d frontend</code>
Parar servicios	<code>docker compose down</code>

Endpoints principales

Endpoint	Función
GET /health	Comprueba API y conexión con PostgreSQL.
GET /info	Devuelve información básica de la aplicación.
GET /metrics	Muestra contadores de eventos, reglas y alertas.
GET /events	Consulta eventos almacenados.

Endpoint	Función
POST /events	Crea eventos de forma directa.
POST /ingest	Recibe eventos y activa el motor de reglas.
GET /rules	Consulta reglas configuradas.
POST /rules	Crea nuevas reglas.
GET /alerts	Consulta alertas básicas.
GET /alerts/ui	Consulta alertas enriquecidas para frontend.
GET /alerts/{alert_id}/ui	Consulta una alerta enriquecida concreta.
GET /alerts/ui/count	Devuelve el número de alertas según filtros.
PATCH /alerts/{alert_id}	Modifica el estado de una alerta.

URLs locales del laboratorio

Servicio	URL
API FastAPI	http://127.0.0.1:8000
Swagger	http://127.0.0.1:8000/docs
Adminer	http://127.0.0.1:8080
Frontend	http://127.0.0.1:5173/index.html
